

微机原理与微系统

第一章 微型计算机概述

微型计算机和处理器的的发展

典型的微型处理器

典型的微型计算机（系统）的基本结构

单片机（Single-chip Microcomputer）/微控制器（Microcontroller）

微型计算机的软件系统

微型处理器的应用

第二章 微型计算机基础知识

微型机中的数制和编码

布尔代数和逻辑电路

常用技术术语

常见技术

冯诺依曼架构

哈佛体系架构

高速缓冲存储器Cache

流水线技术

CISC和RISC

第三章 Introduction to Unix

第四章 单片机的硬件结构

模型机的结构及工作过程

单片机的内部结构

中央处理器

运算器

控制器

数据存储器

RAM

单片机的引脚

1. 电源引脚（Power Pins）

2. 外接晶振引脚 (External Crystal Oscillator Pins)

3. 控制和复位引脚 (Control and Reset Pins)

4. 输入/输出 (I/O) 引脚 (Input/Output Pins)

单片机系统的构成

第五章 数据通信和总线

通信有关的概念

串行通信

串行接口

并行通信

并行接口

串行接口

RS232串行通信接口

RS485串行通信接口

SPI通信接口

第六章 存储器

第一章 微型计算机概述

微型计算机和处理器的的发展

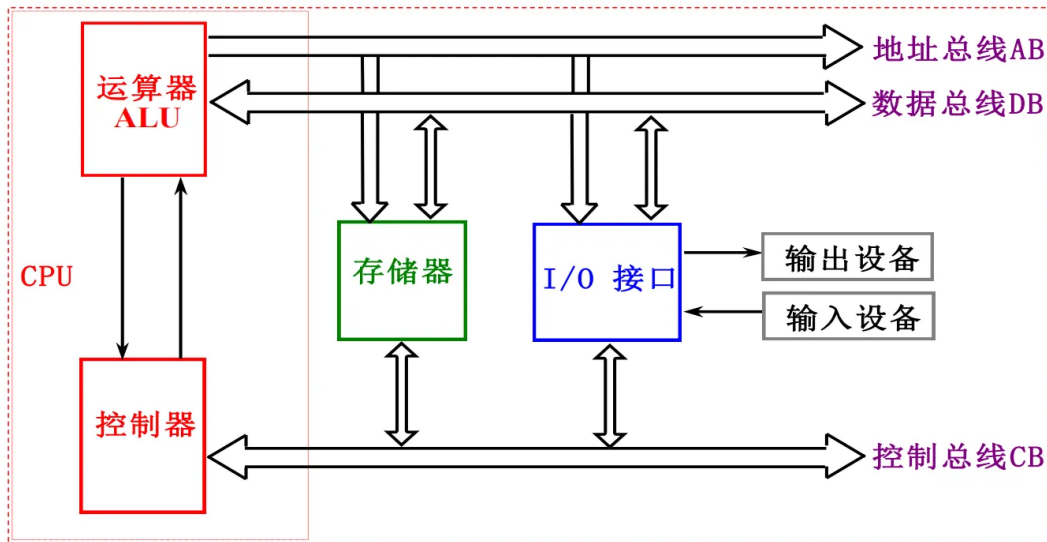
新一代计算机：采用人工智能技术及新型软件，硬件采用新体系结构和超导集成电路，分为问题解决与推理机、知识数据库管理机、智能接口计算机等。具有以下特点：

- 在CPU上集成存储管理部件
- 采用指令和数据高速缓存
- 采用流水线结构以提高系统的并行性
- 采用大量的寄存器组成寄存器堆以提高处理速度
- 具有完善的协处理器接口，提高数据处理能力
- 在系统设计上引入兼容性，实现高、低档微机间的兼容。

典型的微型处理器

典型的微型计算机（系统）的基本结构

- 微处理器（CPU）
- 存储器
- 输入/输出接口（I/O接口）
- 外部设备(输入输出设备)
- 系统总线（地址总线、数据总线、控制总线）



地址总线AB (Address Bus)： 单向，CPU输出的地址信号；

- 输出将要访问的内存单元或I/O端口的地址。
- 地址线的“位 (Bit)”多少决定了系统直接寻址存储器的范围。

数据总线DB (Data Bus)： 双向，数据在CPU与存储器(或I/O接口)间传送；

- CPU读操作时，外部数据
- CPU写操作时，CPU数据
- 数据线的多少决定了一次能够传送数据的位数(8, 16, 32, 64);
- 串行-并行
- CPU通过AB（地址总线）上不同的地址确定读写的存储(接口)单元，经由DB（数据总线）与存储器(或I/O接口)进行数据传输。

控制总线CB (Control Bus)： 双向，CPU对存储器、I/O接口进行控制和联络。

- 输出控制信号: CPU发给存储器或I/O接口的控制信号。如, 微处理器的读信号RD, 写信号WR等。
- 输入控制信号: CPU通过接口接受的外设发来的信号。如, 外部中断请求信号INTR、非屏蔽中断请求输入信号NMI等。
- 控制信号间相互独立，表示方法: 采用能表明含义的缩写英文字母符号。按照惯例，若符号上有一横

线，则表示该信号为低电平有效，否则为高电平有效。

微处理器，简称MP(Micro Processor)，也称 μP ，是微型机的核心部件。通常称为中央处理单元CPU (Central Processing Unit)。

包括：运算器ALU (Arithmetic Logic Unit)、控制器CU (Control Unit)、寄存器阵列R(Registers)、内部总线等电路。集成在一片硅片上。

功能：指令译码，读写数据；响应中断请求；控制。

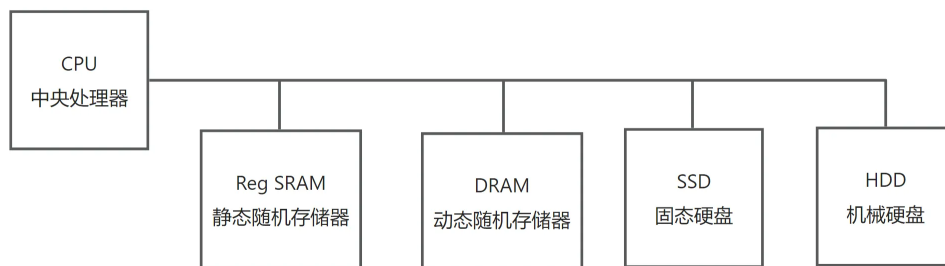
X86、ARM、PowerPC、MIPS.....

存储器

分为程序存储器和数据存储器两类。

片上寄存器（RAM、ROM），内存，硬盘。

根据速度-容量之间的折中，分成多个层次。



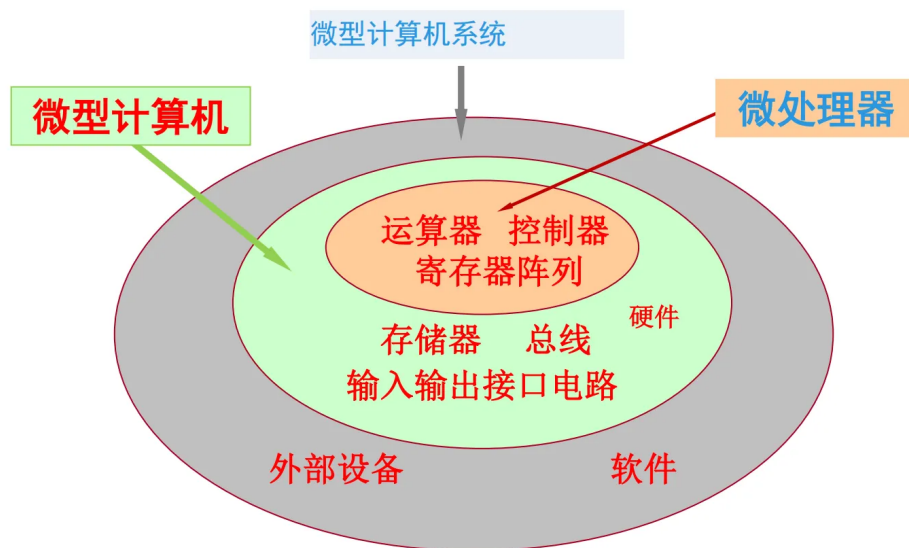
离CPU越近，速度越快，存储空间越小。

对于一个数字系统而言，其功耗大致满足以下公式： $P = CV^2f$ ，其中C为系统的负载电容，V为电源电压，f为系统工作频率。

I/O接口

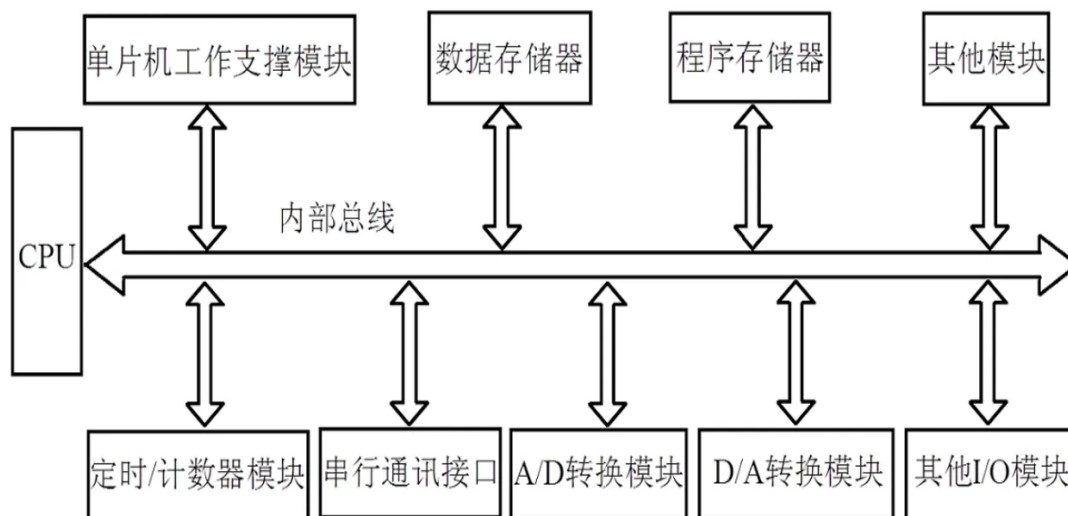
主要用于CPU和外部设备之间交换数据。

并行口、串行口、USB口等。



单片机（Single-chip Microcomputer）/微控制器（Microcontroller）

在一块芯片上集成了中央处理单元(CPU)、存储器(RAM/ROM等)、定时/计数器以及多种输入/输出(I/O)接口的比较完整的数据处理系统。



- 高性能、多功能的方向发展

以个人计算机PC (Personal Computer) 为标志，具有强大的操作系统，并且支持多种软件运行。

- 价格低廉、片上系统（System On Chip, SOC）的方向发展

将CPU、存储器、接口电路、内部总线等部件全部集成在同一个芯片上的单片微机又称为微控制器（Micro-controller），也称为单片机。

微型计算机的软件系统

PC机的运行过程：开机进入系统，执行系统程序，包括开机存储器自检、接口自检、外设自检等等。接受用户通过键盘或者鼠标发出的命令，进一步执行用户要执行的程序。系统程序就把要执行的程序从硬盘里面找到，放进内存，然后运行用户的程序。关闭用户程序时，系统程序会将内存中的信息重新写回到硬盘中保存。

单片机应用系统，可有操作系统(此时一般称之为嵌入式操作系统)的支持，也可没有操作系统的支持。

无论有没有操作系统，用户所编写的应用程序经过编译后都保存在程序存储器中(一般都保存在单片机内部集成的FLASH存储器中)，执行时，由单片机内部的控制器控制程序的执行。

微型处理器的应用

工业生产控制、数据采集和处理、设备控制、智能化仪器仪表、日常生活等

特点：集成度高、体积小、功耗低、可靠性高、使用灵活方便、控制功能强、编程保密化、价格低廉等在制造工业和日用品生产厂家中的微型计算机控制的自动化生产线，为生产能力和产品质量的迅速提高开辟了广阔前景。

第二章 微型计算机基础知识

微型机中的数制和编码

输入接口电路转换十进制与二进制。

十进制转二进制：整数部分转换，除2取余，逆序排列。小数部分转换，乘2取整，顺序排列。

带符号数，最高位（MSB）为符号位，原码中+为0，-为1。原码易于乘除，不易于加减。

反码运算中，符号位运算后如有进位产生，则把这个进位送回到最低位去相加，这叫循环进位。

正数反码补码不变；负数反码除符号位外全部翻转，补码为反码+1。

$$[[X]_{反}]_{反} = [X]_{原}, [[X]_{补}]_{补} = [X]_{原}$$

无符号数加减运算可能出现溢出，造成溢出错误。补码可以直接舍去溢出的进位。

▼ 加法器可以完成所有运算

加法算减法：因为减去一个正数的减法运算可以看作是加上一个负数的加法运算，所以在计算机中，求得补码之后，就把减一个正数的运算转变为加上该负数的补码的加法运算。

加法算乘法：可以采用移位相加的方法完成。

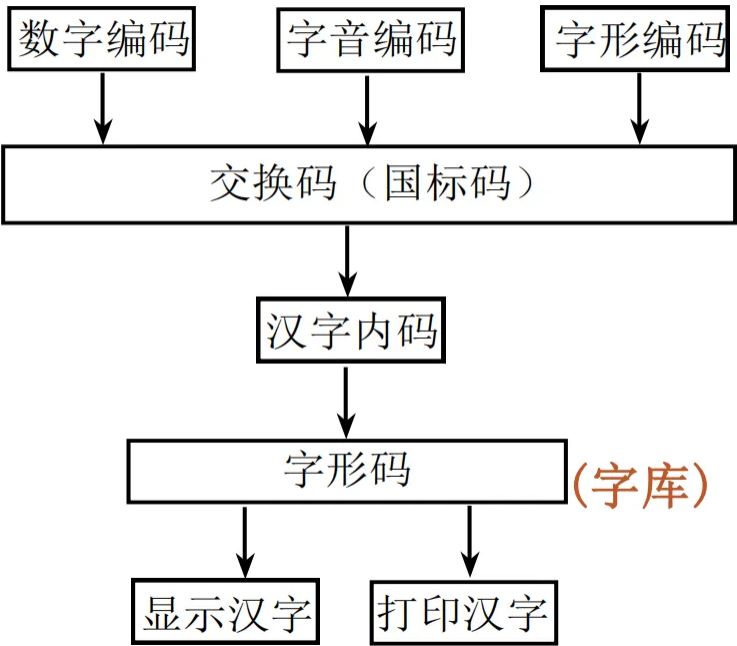
加法算除法：采用移位相减的方法完成，这样只用加法器就能完成所有的算术运算。

8421BCD码：按照8,4,2,1权值。非压缩BCD，8位表示0–9，高四位为零；压缩BCD，8位表示0–99，高四位表示十位，低四位表示个位。

国际上通用的标准字符编码为ASCII码 (American Standard Code for Information Interchange, ASCII)，即美国标准信息交换码。0–31及127(del)(共33个)为控制字符(ASCII control characters)。32(空格)–126为可打印字符(ASCII printable characters)。第八位为奇偶校验位，通过对奇偶校验位设置“1”或“0”状态，保持8位字节中的“1”的个数总是奇数(称为奇校验)或偶数(称为偶校验)，一般用于字符或数字的串行传送时检测传送过程中是否出错。

汉字编码中，常用输入编码有数字、字音、字形和音形编码等。以国家标准局GB2312–80规定的汉字交换码作为标准汉字编码。共收录7445个。汉字区位码：在字符集中,汉字和字符分94个区,每区94位。每个汉字及字符用两个字节表示，前一字节为区码，后一字节为位码，各用两位16进制数字表示。国标码 = 区位码（化成16进制）+ 2020H。汉字内码用两个字节表示：为区分汉字与英文字符，将汉字国标码每字节的最高位置1，即汉字机内码。汉字机内码 = 汉字国标码 + 8080H。

(区位码)



统一代码（Unicode）是一种全新的编码方法，此法有足够的能力来表示全世界多达6 800种语言中任意一种语言里使用的所有符号。其基本方法是，用1个16位的数来表示Unicode中的每个符号，即允许表示65 536个不同的字符或符号。这种符号集被称为基本多语言平面（BMP）。

布尔代数和逻辑电路

运算规则：

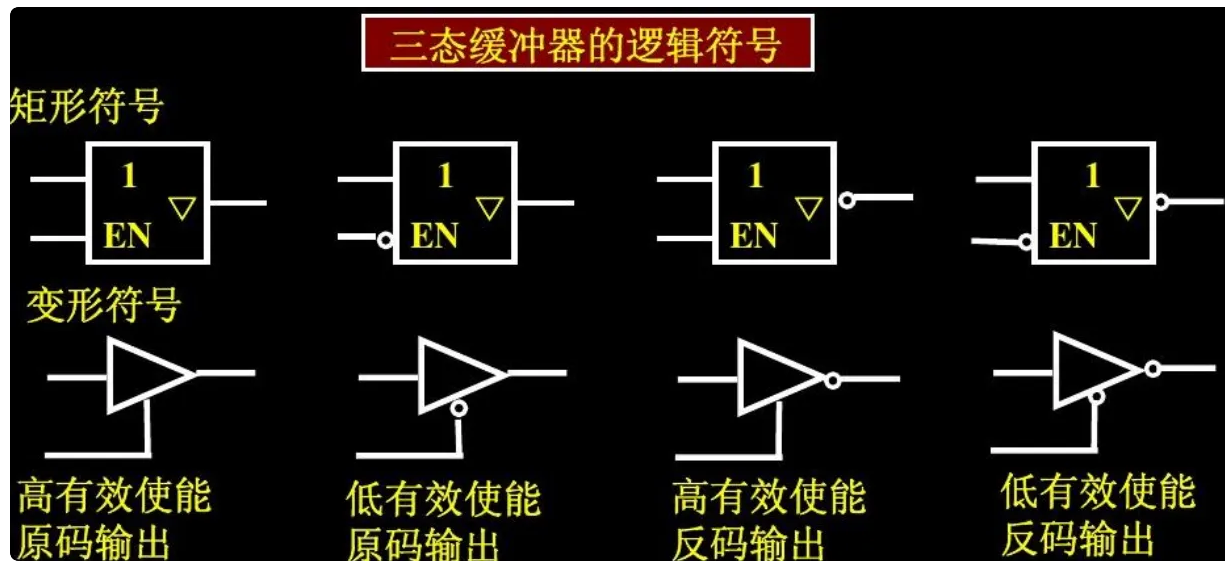
吸收律： $A + A \cdot B = A$, $A \cdot (A + B) = A$

第二吸收律: $A + \bar{A} \cdot B = A + B$, $A \cdot (\bar{A} + B) = A \cdot B$

包含律: $AB + \bar{A}C + BC = AB + \bar{A}C$, $(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$

组合逻辑电路:

三态门



锁存器

- D0~D7 为数据输入端
- OC为输出允许控制端(三态, 低电平有效)
- G为锁存允许端(也称为LE端)
- Q0~Q7为输出端

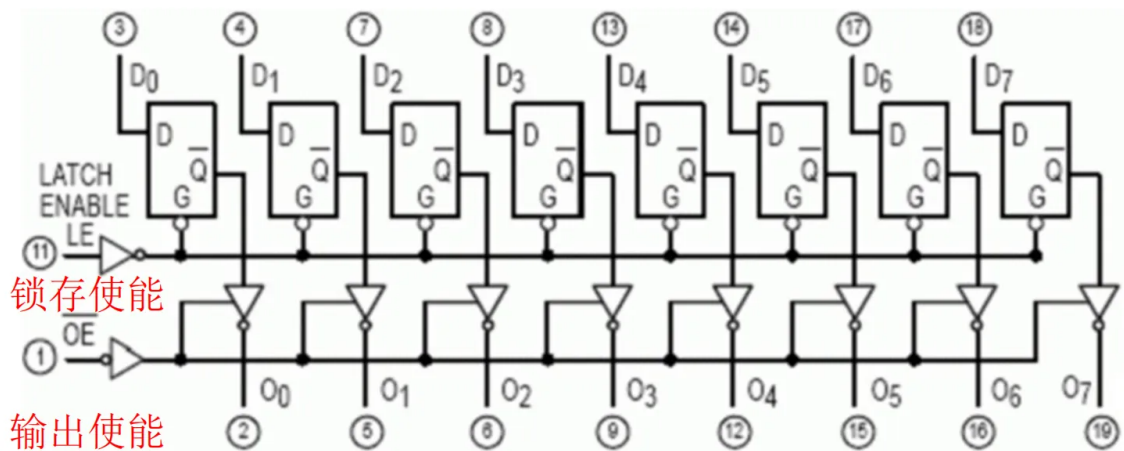
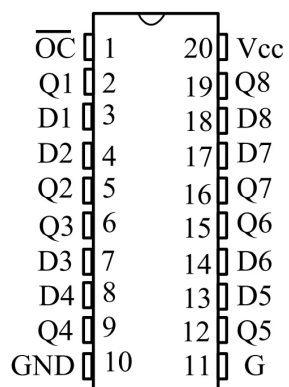
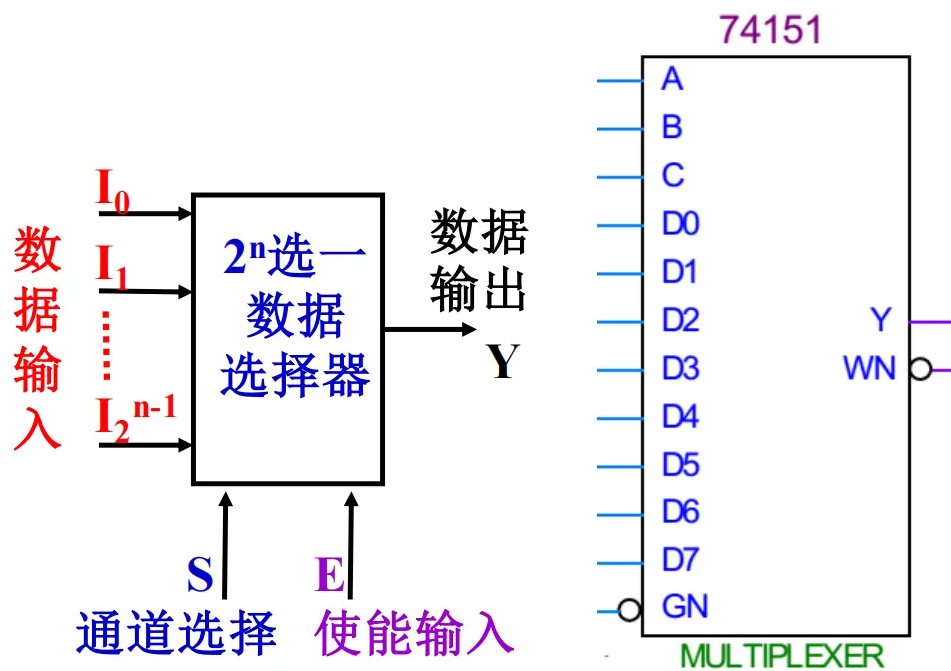


图2-7 74LS373内部逻辑电路图



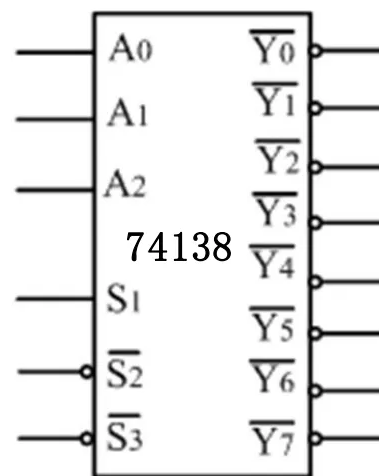
数据选择器



译码器

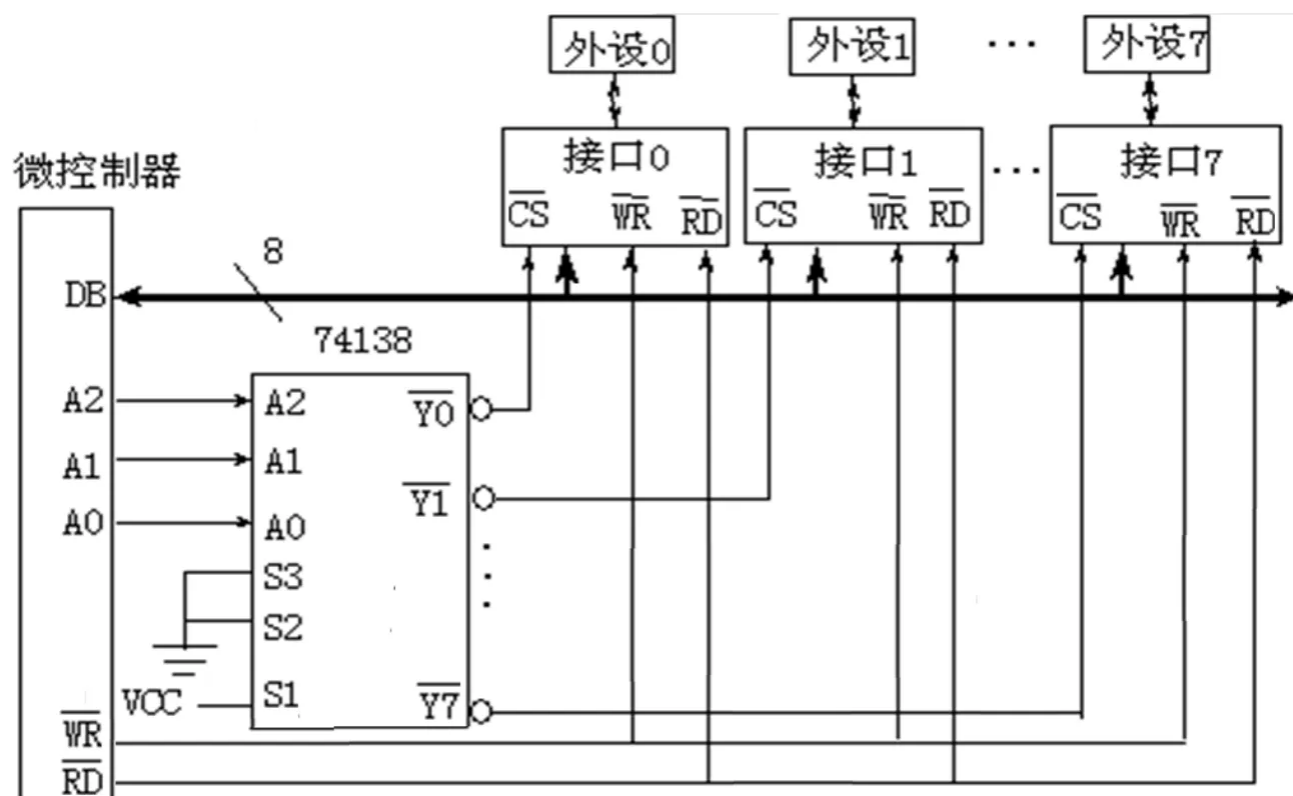
74138是具有3个输入、8个输出的集成译码器，就是通常所说的三八译码器。73LS138芯片是TTL电平，74HC138是CMOS电平的。

输 入					输 出							
S_1	$\overline{S_2} + \overline{S_3}$	A_2	A_1	A_0	$\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$
0	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0



片选端CS/CE，选择芯片或使能芯片。

通过地址译码器与片选端连接，使微控制器工作。



常用技术术语

1. 位bit：计算机最小数字单位，常用b表示
2. 字节byte：8位为1字节，内存基本单位，常用B表示
3. 字word：16位=2字节=1字

4. 字长：字长即字的长度, 是一次可并行处理数据的位数即数据线的条数。常与CPU内部寄存器, 运算装置, 总线宽度一致。常用微机字长有8位, 16位和32位。
5. 数量单位, $kilo : 1K = 2^{10}$, $maga : 1M = 2^{20}$, $giga : 1G = 2^{30}$, $tera : 1T = 2^{40}$
6. MIPS: 单字长定点指令平均执行速度Million Instructions Per Second的缩写, 即每秒处理百万条机器语言指令数。这是衡量CPU速度的一个指标。
7. 地址: 存储单元的编号, 8bit为一个单元。存储器地址的最大编号(容量)有限, 由地址线的条数决定。
8. 总线: CPU与存储器、IO口通过三总线联系。
9. 访问: 写入操作&读出操作
10. 机器指令: 由二进制代码组成, 直接由微处理器译码和执行。一条机器指令应包含要求微处理器所要完成的操作, 以及参与该操作的数据或该数据所在的地址, 有时还要有操作结果的存放地址信息。
11. 汇编指令: 用英文缩写表示, 不同厂家的CPU配有不同格式的汇编语言指令系统, 互不兼容, 用汇编语言编写的程序叫做汇编语言源程序。
12. 指令系统: 一台计算机能识别的全部指令的集合。
13. 汇编: 汇编指令→机器指令; 反汇编: 机器指令→汇编指令。
14. 高级语言: 一般用高级语言编程, 再用某种特殊程序翻译成机器语言。编译器将用高级语言编写的用户程序翻译成某个具体的微处理器的机器语言程序。C编译器, 能把C语言转换成某个具体的微处理器的机器语言。用汇编语言编程能更有利于硬件电路与程序的结合设计与调试。高级语言程序冗长、不够简练, 运行时时间长, 速度低。

常见技术

冯诺依曼架构

计算机系统由一个中央处理单元(CPU)和一个存储器组成, 数据和指令都存储在存储器中, CPU可以根据所给的地址对存储器进行读或写。程序指令和数据的宽度相同。

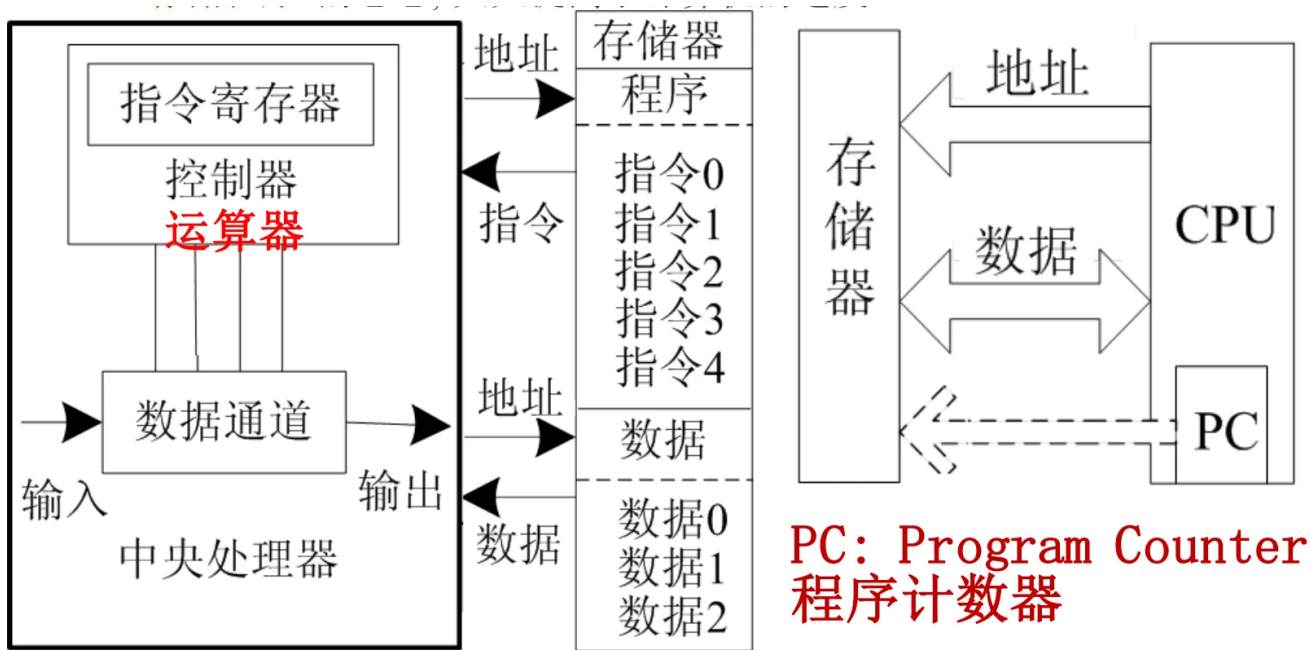


图2-12 冯·诺依曼结构的构成示意图

- 。计算机包括运算器、控制器、存储器、输入/输出设备。
- 。计算机内部应采用二进制来表示指令和数据。
- 。将编写好的程序和数据保存到存储器，然后计算机自动地逐条取出指令和数据进行分析、处理和执行。

哈佛体系架构

哈佛体系结构中, 数据和程序使用各自独立的存储器。程序计数器PC只指向程序存储器而不指向数据存储器。

独立的程序存储器和数据存储器为数字信号处理提供了较高的性能。

指令和数据可以有不同的数据宽度,具有较高的效率。

很难在哈佛体系结构的计算机上编写出一个自修改的程序（在应用可编程IAP）。

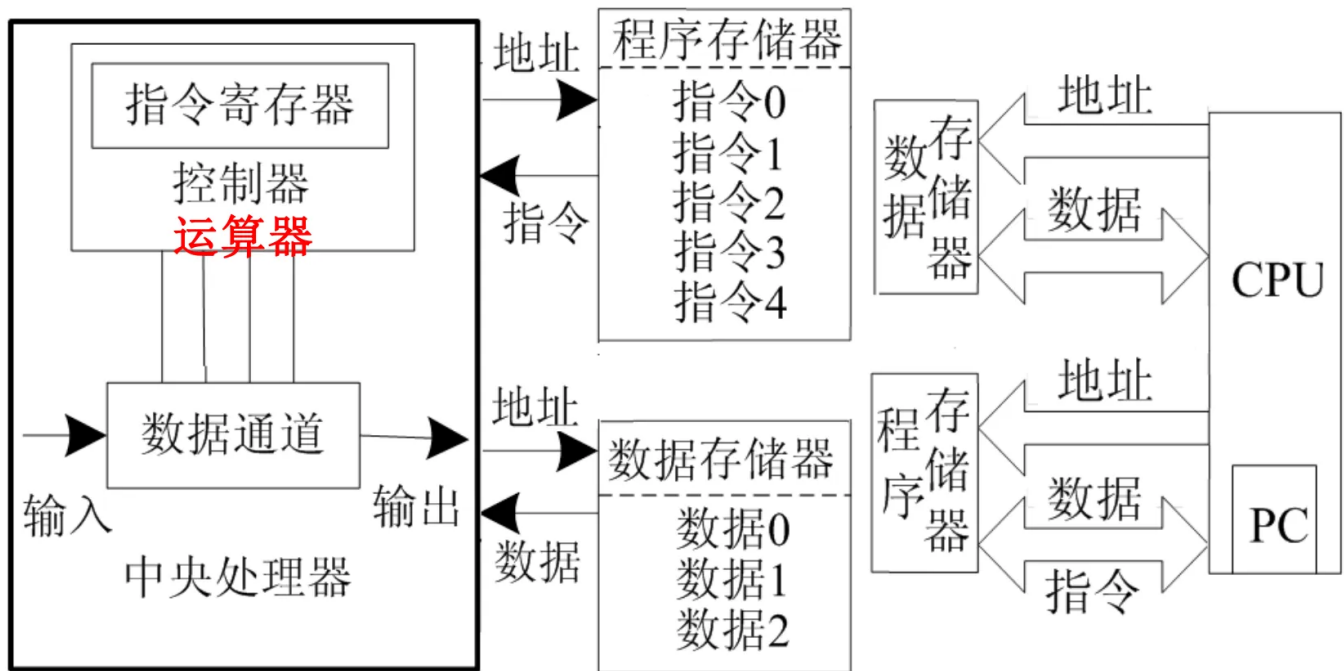


图2-13 哈佛结构的构成示意图

高速缓冲存储器Cache

片内/片外，Cache采用与制作CPU相同的半导体工艺，速度与CPU匹配。当CPU要从主存储器(个人计算机中称内存)中读取一个数据时，先在Cache中查找是否有该数据，若有，则从Cache中读取到CPU，否则从主存储器中读取这个数据送到CPU，与此同时将包含这个数据字的整个数据块送到Cache中。只要算法策略得当，Cache的命中率可达99%以上，它有效地减少CPU访问低速内存的次数，从而提高读取数据的速度和整机的性能。

流水线技术

程序执行时多条指令重叠进行操作的一种准并行处理实现技术。提高CPU效率。

在CPU中由5~6个不同功能的电路单元组成一条指令处理流水线，然后将一条X86指令分成5~6步后再由这些电路单元分别执行，这样就能实现在一个CPU时钟周期完成一条指令，因此提高CPU的运算速度。

CISC和RISC

CISC：复杂指令集计算机 (Complex Instruction Set Computer),使用大量的指令集,包括复杂指令,可编写短程序,完成复杂功能。由于指令的复杂,增加了处理器的结构复杂性以及逻辑电路的级数,降低了时钟频率,使指令执行的速度变慢,纯CISC结构的处理器执行一条指令至少需要一个以上的时钟周期。

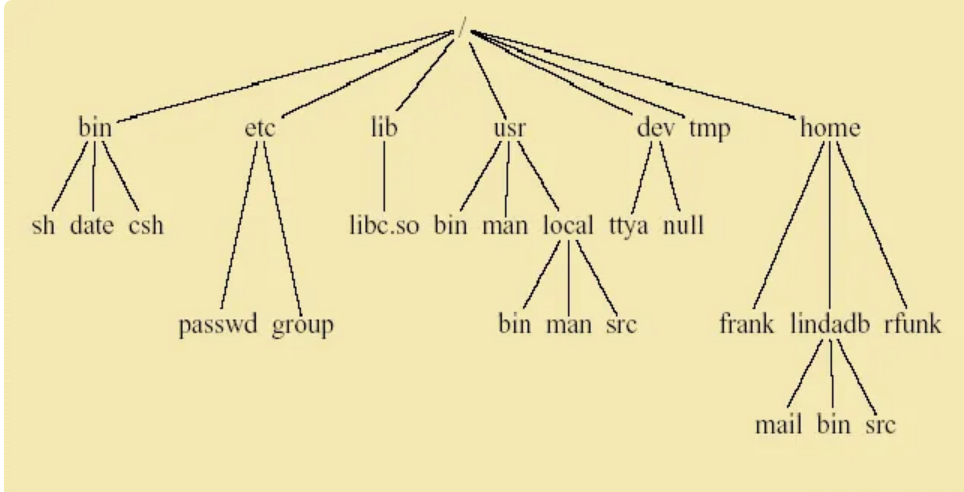
RISC: Reduced Instruction Set Computer, 精简指令集计算机的简称。其指令集结构只有少数简单的指令, 使计算机硬件简化, 将CPU的时钟频率提高, 配合流水线结构可做到一个时钟周期执行一条指令, 使整个系性能超过CISC结构的计算机。选取使用频率最高的一些简单指令; 指令长度固定, 指令格式和寻址方式种类少; 只有取数据和存数据指令访问存储器, 其余指令的操作数在寄存器之间进行。

第三章 Introduction to Unix

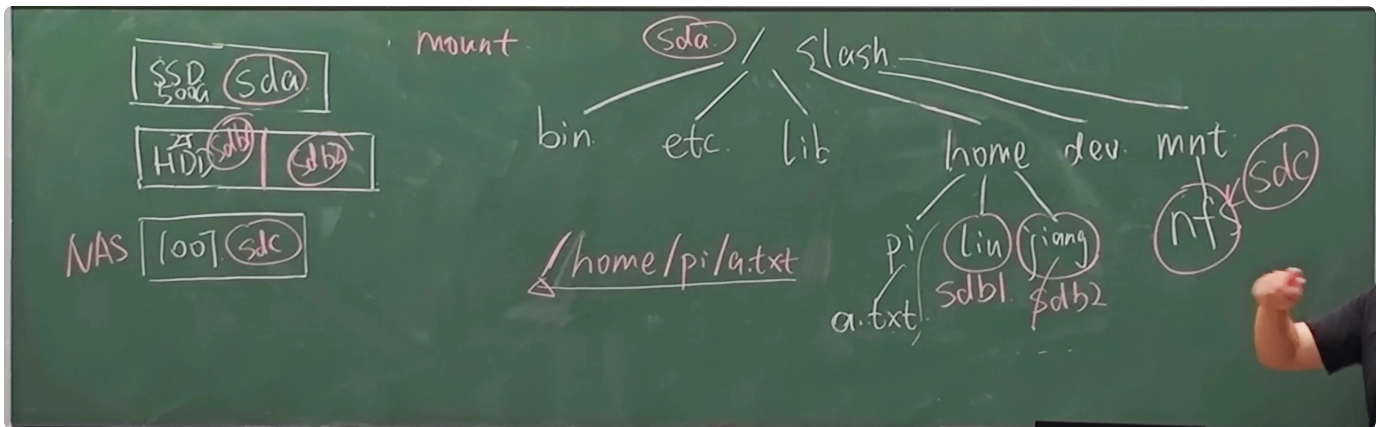
Linux Philosophy

- Multiuser / Multitasking
- Toolbox approach (make each do one thing well)
- Flexibility / Freedom
- Conciseness (small is beautiful)
- Everything is a file
- File system has places, process has life
- Designed by programmers for programmers, Even of the programmers

File System:



home挂载在sdb下, 其他文件挂载在sda下, 可以再有一个分支连到挂载在sdc的网盘中



A program or command interacts with the kernel may be any of:

- built-in shell command
- interpreted script
- compiled object code file

执行: command options arguments

Control keys (^) perform special functions

- ^H or Backspace erase a character
- ^U cancel line
- ^S pause display
- ^Q restart display
- ^C cancel operation
- ^D signal end of file
- ^V treat following control character as normal character

alias aname pname定义别名

添加环境变量

追加是: `export PATH=新路径:$PATH`

覆盖是: `export PATH=新路径`

```

[jiangjunmin@geex001 ~]$ ls -l
total 2292
-rw-r--r--. 1 jiangjunmin jiangjunmin 1262 Jul  6 13:59 bashrc.usr
-rw-rw-r--. 1 jiangjunmin jiangjunmin 75739 Aug  2 13:24 CDS.log
-rw-rw-r--. 1 jiangjunmin jiangjunmin 2246498 Jul 16 13:04 CDS.log.1
drwxrwxr-x. 7 jiangjunmin jiangjunmin 74 May 11 19:02 cqserver
drwxr-xr-x. 2 jiangjunmin jiangjunmin 6 Apr 30 16:31 Desktop
drwxr-xr-x. 10 jiangjunmin jiangjunmin 210 Sep  8 19:19 Downloads
drwxr-xr-x. 27 root      root      4096 May 11 14:48 jiangjm
-rw-r--r--. 1 jiangjunmin jiangjunmin 467 Jul  9 01:11 Makefile
drwxrwxr-x. 6 jiangjunmin jiangjunmin 92 Jul 21 21:47 project
drwxrwxr-x. 21 jiangjunmin jiangjunmin 4096 Aug  1 22:41 simulation
-rwxr-xr-x. 1 jiangjunmin jiangjunmin 40 May 11 19:17 vncrun
  
```

权限，用户，大小（字节），修改日期，文件类型

权限分三个组，user(u)，group(g)，others(o)

- r read 4
- w write 2
- x excute 1
- – no permission 0

chmod 777 file/ chmod u+x file

umask 022——default mod 755

第一个字，表示文件类型

- **d**：表示目录（directory）。
- **-**：表示普通文件。
- **l**：表示符号链接（symbolic link）。
- **b**：表示块设备文件（block device）。
- **c**：表示字符设备文件（character device）。
- **s**：表示套接字（socket）。
- **p**：表示命名管道（FIFO）。

展示命令：

- echo echo the text string to stdout
- cat concatenate (list)
- head display first 10 (or #) lines of file
- tail display last 10 (or #) lines of file
- more browse or page through a text file

ln: link, symbolic link, hard link, 一般不做hard link, 否则删除会直接删除源文件

unlink, 删除链接

gzip压缩

wc, display count of lines

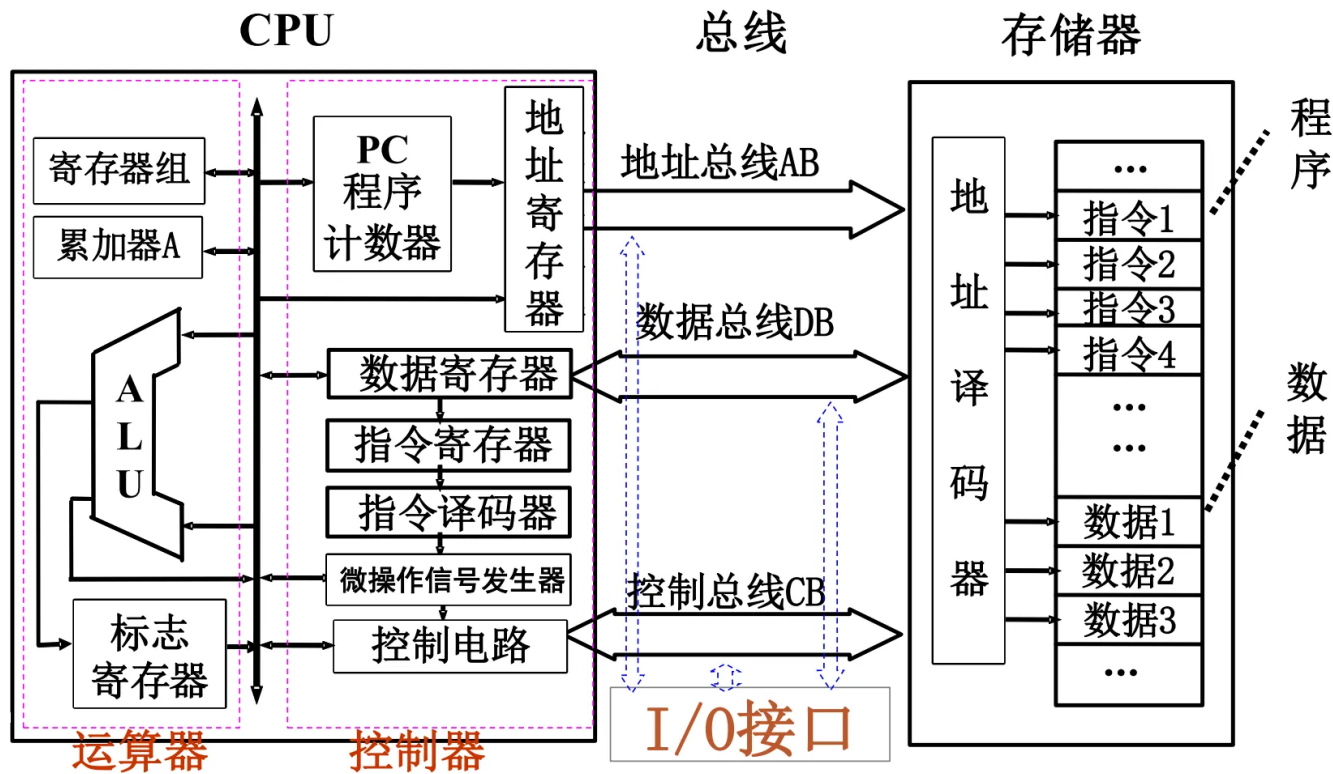
diff file1 file2按行查找区别

grep/egrep/fgrep – search the argument for all occurrences of the search string (get REP)

第四章 单片机的硬件结构

模型机的结构及工作过程

三总线、CPU、存储器、IO接口



CPU中央处理器：

存储器：地址译码器，存储单元，控制逻辑

(1) 读操作：CPU首先将地址寄存器AR的内容放到地址总线AB上，地址总线上的内容进入地址译码器，由地址译码器进行译码，选通相应的存储单元。被选通的存储单元的内容就出现数据总线上，在控制信号的作用下，CPU从数据总线上读取数据到数据寄存器DR，从而完成存储器的读操作。

(2) 写操作：CPU将地址寄存器AR的内容送到地址总线AB上，地址总线上的内容进入地址译码器，由地址译码器进行译码，以选通相应的存储单元。在控制信号的作用下，CPU将要写入的数据通过数据总线写入到被选通的存储单元，完成存储器的写操作。

执行过程：读取指令→分析指令→执行指令→保存结果

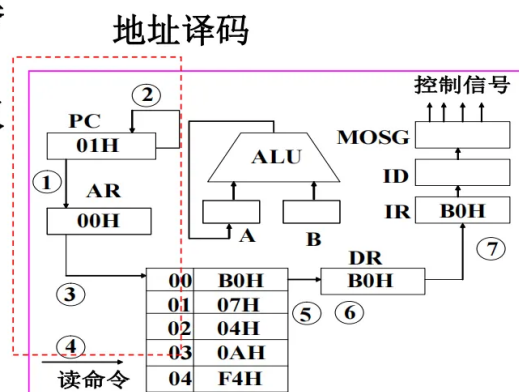
例如, 计算 $7+10=?$, 结果在A中。

假设程序在存储器中的**存储格式**(设程序从00H开始存放)如图示。

地址	存储内容	机器码
00H	B0H	B0H 07H
01H	07H	04H 0AH
02H	04H	F4H
03H	0AH	
04H	F4H	

读取指令阶段的执行过程如下:

- ①CPU将**程序计数器PC**的内容00H送地址寄存器AR。
- ②程序计数器PC的内容自动加1变为01H, 为取下一条指令作好准备。
- ③地址寄存器AR将00H通过地址总线AB送至存储器**地址译码器**译码, 选中00H单元。
- ④CPU发出“读”命令。
- ⑤所选中00单元的内容B0H由存储器送至数据总线DB上。
- ⑥经数据总线DB, CPU将读出内容B0H送数据寄存器DR。



- ⑦数据寄存器DR将其内容送指令寄存器IR中, 经过译码, CPU“识别”出此操作码为**两字节指令的第一个字节**,再取出下一个字节(机器码)后得知是“MOV A, #07H”指令, 于是控制器发出**执行这条指令**的控制命令。

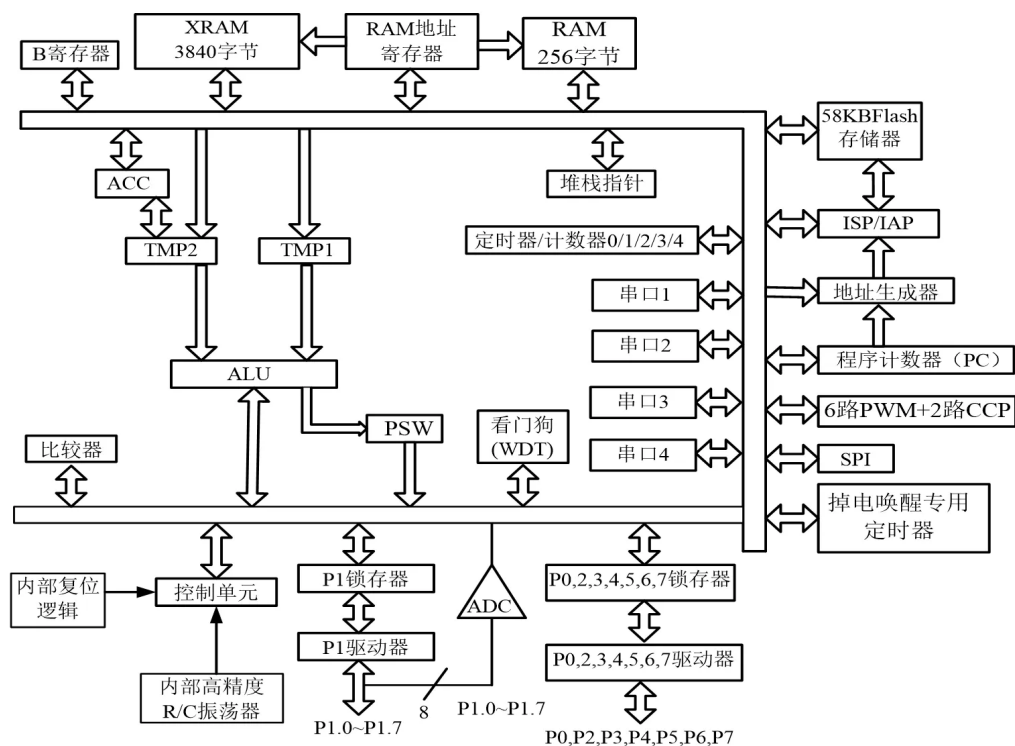
gcc工作过程:

1. 预处理: gcc -E hello.c -o hello.i
2. 编译: gcc -S hello.i -o hello.s
3. 汇编: gcc -c hello.s -o hello.o或者 as hello.s -o hello.o
4. 链接: 链接动态库和静态库后, 生成可执行文件

单片机的内部结构

有些单片机不仅集成了CPU、存储程序和数据存储器、系统总线、I/O接口、定时/计数器 etc 常规资源, 而且还集成了工业测控系统中常用的模拟量采集模块。

IAP15W4K58S4几乎包含了数据采集和控制中所需的所有单元模块, ——可称得上一个片上系统(SOC)



中央处理器

单片机的中央处理器（CPU）由运算器和控制器组成。

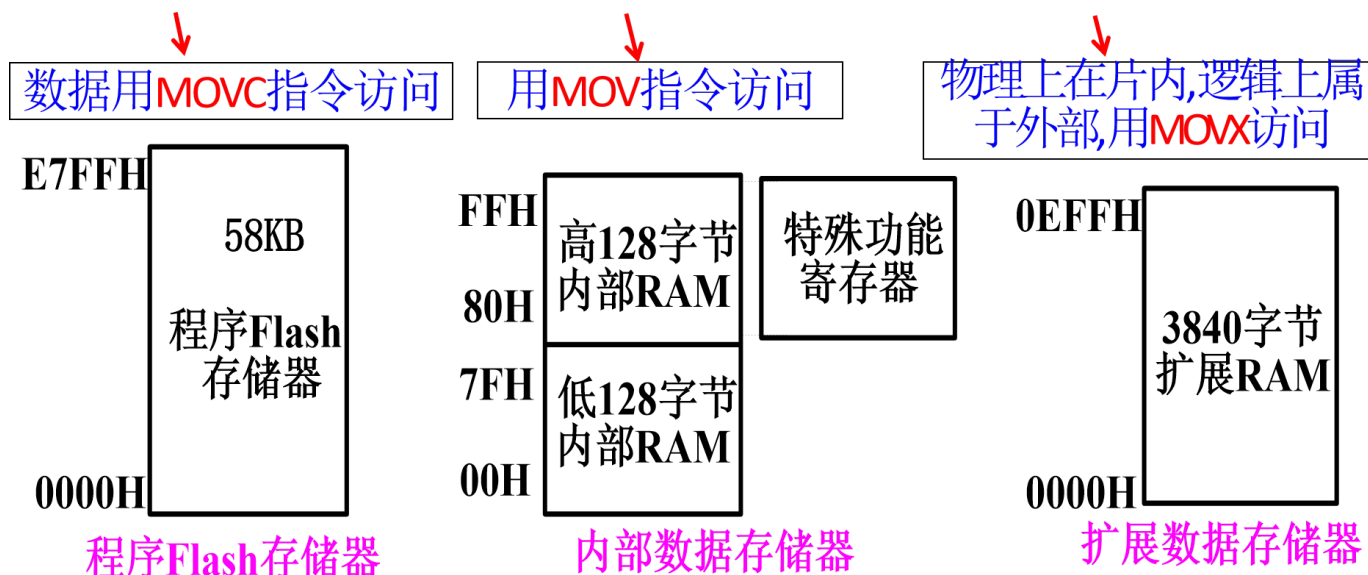
运算器

以8位算术/逻辑运算部件ALU为核心，加上通过内部总线而挂在其周围的暂寄存器TMP1、TMP2、累加器ACC、寄存器B、程序状态标志寄存器PSW以及布尔处理机组成了整个运算器的逻辑电路。

控制器

CPU的大脑中枢, 包括定时控制逻辑、指令寄存器、译码器、(数据)地址指针DPTR及程序计数器PC、堆栈指针SP、RAM地址寄存器等。

数据存储器



- **程序Flash存储器**存储的是代码和固件，是系统不可修改的核心部分之一。
- **内部数据存储器**是用于运行时数据处理的内存，如局部变量、栈空间等。它通常是易失性的，在断电后数据会丢失。
- **扩展数据存储器**则是用来增加系统的数据存储能力，通常是外部设备，具有更大的容量，可用于存储大量数据或永久数据。

RAM

数据存储器也称为随机存取数据存储器(RAM)。

- **内部RAM (Internal RAM) 功能：**内部RAM是微控制器内部集成的随机存取存储器，用于存储程序运行时的临时数据，如变量、栈、堆等。
- **XRAM (扩展RAM, External RAM) 功能：**XRAM指的是外部的随机存取存储器，通常连接到微控制器的外部存储设备。它用于扩展内存容量，以满足更大数据量存储的需求。

单片机的引脚

1. 电源引脚 (Power Pins)

电源引脚是为微控制器提供工作电压的引脚，通常包括：

- **Vcc (或Vdd) 引脚：**通常是微控制器的正电源引脚，连接到系统的正电源（例如5V或3.3V）。
- **GND引脚：**地引脚，用于连接到电源的地线（负极）。
- **Vref引脚 (可选)：**一些微控制器有参考电压引脚，用于设置ADC（模拟-数字转换器）的参考电压，确保模拟输入的准确性。

电源引脚通常还可能有一些特殊功能，例如：

- **低功耗模式引脚**：用于切换微控制器进入低功耗模式或正常模式。
- **过压保护引脚（可选）**：用于防止电压过高对微控制器造成损坏。

2. 外接晶振引脚（External Crystal Oscillator Pins）

外接晶振引脚用于连接外部的晶体振荡器（或陶瓷振荡器）来提供微控制器的时钟信号。微控制器需要一个稳定的时钟源来驱动其时序和操作。外接晶振引脚通常包括：

- **XIN、XOUT引脚**：这些是连接外部晶振的引脚，XIN为输入引脚，XOUT为输出引脚，通常连接到晶体振荡器。
- **外部时钟引脚（如果没有内部振荡器）**：有些微控制器允许通过外部时钟信号输入，代替内部时钟源。此时，时钟信号输入到一个专门的引脚上。

如果微控制器有内部时钟源（例如内部RC振荡器），外部晶振引脚则可能被用作备用时钟输入或配置引脚。

3. 控制和复位引脚（Control and Reset Pins）

控制和复位引脚负责微控制器的启动、复位以及模式切换。常见的控制和复位引脚有：

- **复位引脚（RESET引脚）**：用于将微控制器复位到初始状态。按下复位按钮或通过外部电路施加一个低电平信号，微控制器将重新启动，程序从起始位置开始执行。复位过程通常包括清除RAM内容、重新初始化外设等。
- **外部中断引脚（如INT0、INT1等）**：用于接收外部中断信号，触发微控制器中断服务程序（ISR）执行。外部中断引脚一般用于响应外部事件（如按键按下、传感器信号等）。
- **低功耗引脚（如SLEEP引脚）**：用于控制微控制器的低功耗模式。当微控制器进入低功耗模式时，该引脚可以被用于唤醒系统或恢复正常操作。

4. 输入/输出（I/O）引脚（Input/Output Pins）

I/O引脚是微控制器与外部世界进行数据交换的接口，支持数字或模拟信号的输入输出。根据功能和配置方式，I/O引脚可分为以下几类：

数字I/O引脚：

- **输入引脚**：用于读取外部信号，如按键、传感器或其他数字设备的输出。
- **输出引脚**：用于向外部设备发送信号，如驱动LED、控制继电器、控制外部设备等。
- **双向I/O引脚**：可以配置为输入或输出模式，允许更灵活的接口设计。

模拟I/O引脚：

- **模拟输入引脚（ADC引脚）**：用于从模拟信号源读取模拟电压，并通过内部的模拟-数字转换器（ADC）将其转换为数字值。例如，可以连接温度传感器、光传感器等模拟设备。
- **模拟输出引脚（DAC引脚）**：一些微控制器也提供模拟输出功能，通常通过数字到模拟转换器（DAC）将数字信号转换为模拟电压信号。模拟输出在音频、信号生成等应用中非常有用。

特殊功能I/O引脚：

一些微控制器的I/O引脚具有额外的功能，例如：

- **PWM输出引脚**：用于产生脉宽调制信号（PWM），可以控制电机速度、亮度调节（如调节LED亮度）等。
- **串行通信引脚**（如UART、SPI、I2C引脚）：用于与其他设备进行串行通信，如传感器、LCD显示器、外部存储器、无线模块等。常见的串行通信引脚有：
 - **UART引脚**：TX（发送）、RX（接收）。
 - **SPI引脚**：MOSI（主设备输出，外设输入）、MISO（主设备输入，外设输出）、SCK（时钟）、CS（片选）。

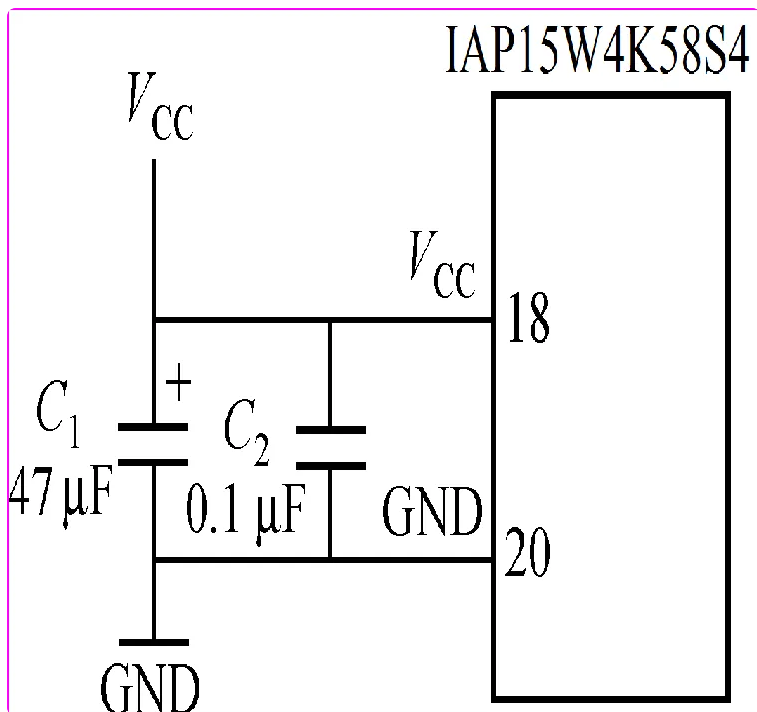
I2C引脚：SCL（时钟线）、SDA（数据线）。

PnM1 [7:0]	PnM0 [7:0]	(n=0, 1, 2, 3, 4, 5、6、7) I/O口工作模式
0	0	准双向口 (传统8051单片机I/O口模式), 灌电流可达 20mA ，拉电流为 270μA ，由于制造误差，实际为 150uA~270uA
0	1	推挽输出 (强上拉输出, 可达 20mA , 要加限流电阻, 尽量少用)
1	0	仅为 输入 (高阻)
1	1	开漏(Open Drain) , 内部上拉电阻断开, 要外加上拉电阻

单片机系统的构成

最小系统

IAP15W4K58S4集成了58KB程序存储器、4096字节RAM、高可靠复位电路和高精度R/C振荡器,一般不需外部复位电路和外部晶振。只需要接上电源, 并在Vcc和GND之间接上滤波电容C1和C2。



总线扩展式单片机系统 (Bus-Extended MCU System)

总线扩展式单片机系统是通过总线（如地址总线、数据总线和控制总线）与多个外部设备进行连接的系统。外设与单片机共享一条总线，通过总线进行数据的传输与访问。其特点包括：

第五章 数据通信和总线

通信有关的概念

并行通信：以字节(Byte)或字节的倍数为传输单位;一次传送一个(以上)字节的数据，数据各位同时传送。近距离、大量、快速的数据交换。

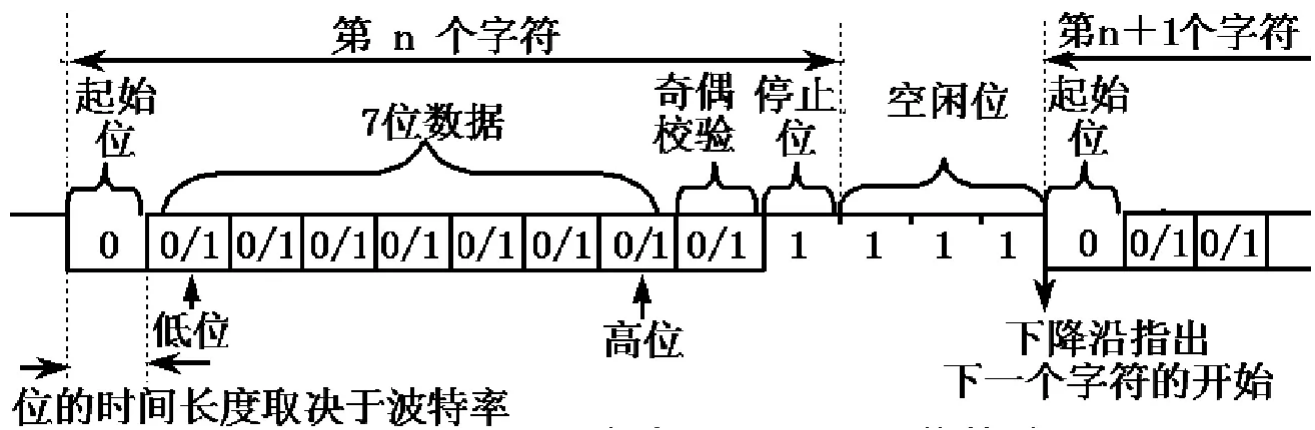
串行通信：用2(或3)根数据信号线相连, 数据在一根数据信号线上一位一位地顺序传送, 每位数据都占据一个固定的时间长度。传输线少、成本低、适合远距离传送及易于扩展。

串行通信

异步/同步：

- 异步通信：数据传送以字符为单位, 字符与字符间的传送是完全异步的, 位与位之间的传送基本上是同步的。异步传送中,每个字符要用起始位和停止位作为字符开始和结束的标志, 以字符为单位逐个发送和接收。需要规定好：字符格式：包括字符的编码形式、奇偶校验以及起始位和停止位的规定。

通信速率：通信速率通常使用比特率来表示。

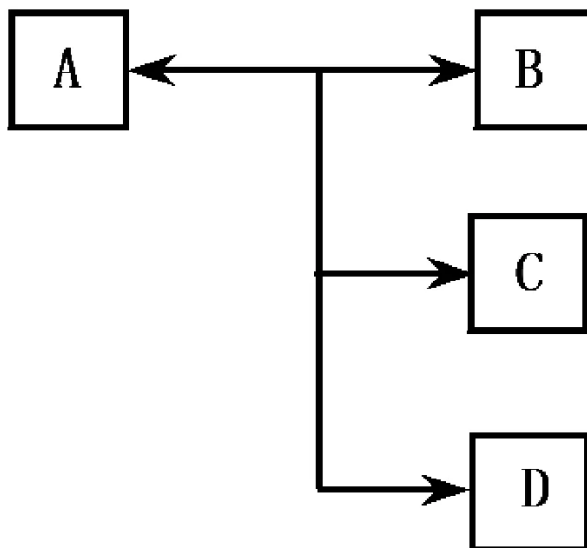


- 同步通信(Synchronous Communication)是一种连续串行传送数据的通信方式，一次通信只传送一帧信息。传输速率较高，适用于传送信息量大、传送速率高的系统中。要求发送时钟和接收时钟保持严格同步，故发送时钟除应和发送波特率保持一致外，还要求把它同时传送到接收端去。

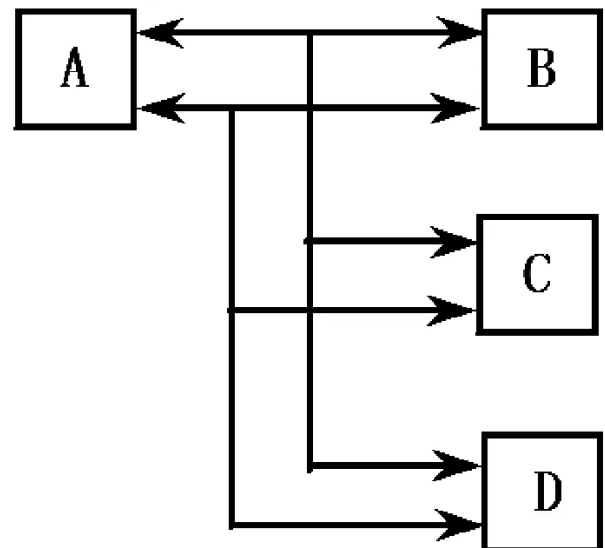
单工/半双工/全双工

- 单工通信方式 (Simplex)。A为发送站，B为接收站，数据只能由A发至B，而不能由B传送到A。
- 半双工通信方式 (Half Duplex)。数据可从A发送到B，也可由B发送到A。不过，由于使用一根线连接，发送和接收不可能同时进行，同一时间只能作一个方向的传送，其传送方向由收发控制开关K来控制。
- 全双工通信方式 (Full Duplex)。在这种方式中，分别用2根独立的传输线来连接发送方和接收方，A、B既可同时发送，又可同时接收。

主从多终端通信方式。A可以向多个终端(B, C, D...)发出信息。在A允许的条件下，可以控制管理B, C, D等在不同的时间向A发出信息。又分为多终端半双工通信和多终端全双工通信。



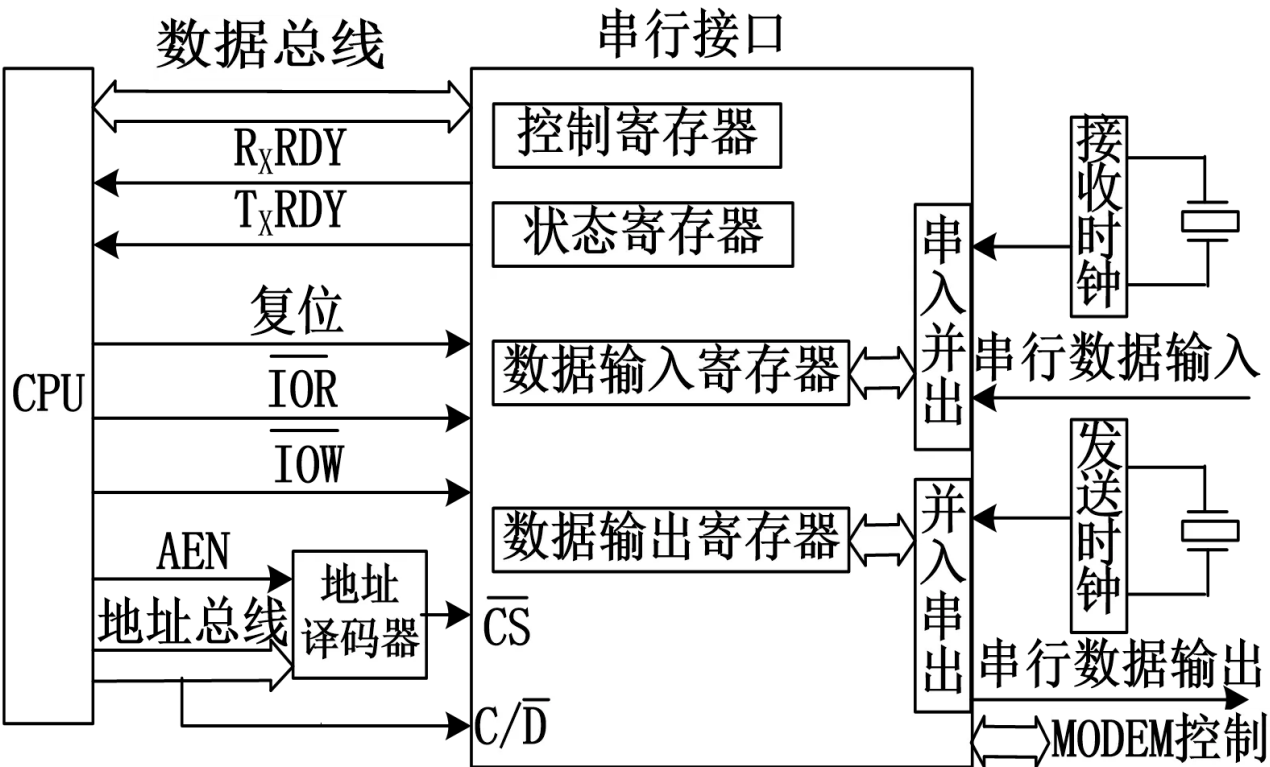
a) 多终端半双工通信方式



b) 多终端全双工通信方式

串行接口

串行通信中的数据是一位一位依次传送的，而计算机中数据是并行传送的。因此，发送端必须把并行数据变成串行才能传送，接收端接收到的串行数据又需要变换成并行数据才可以送给计算机。



串行接口主要由4部分组成：

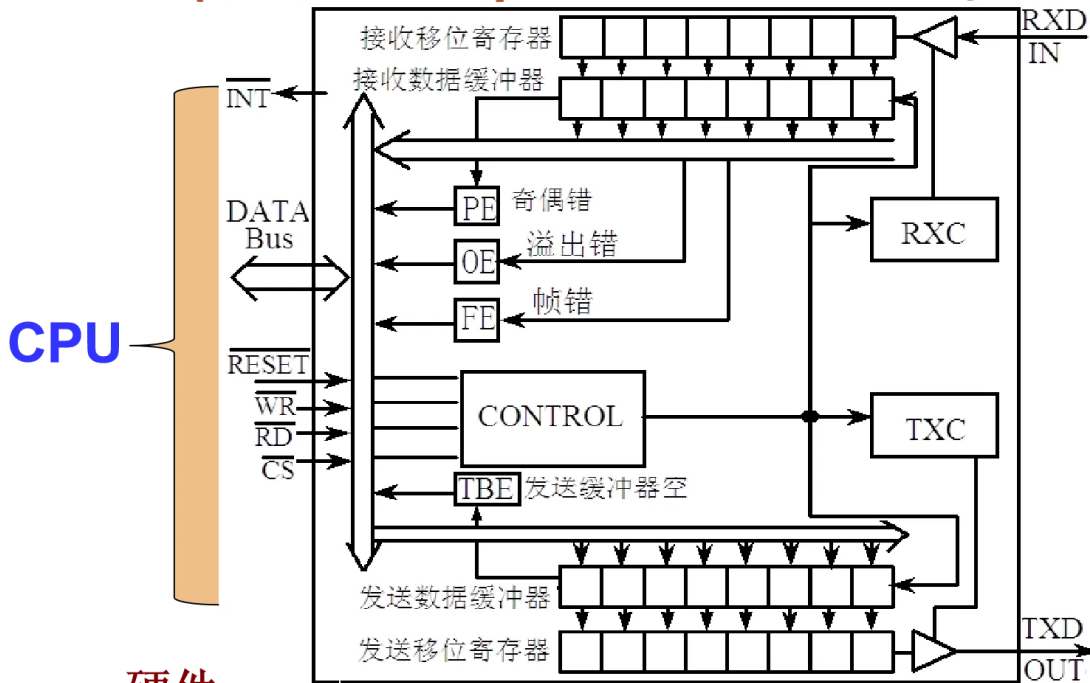
- 数据输入寄存器。在输入过程中，串行数据一位一位地从传输线进入串行接口的接收移位寄存器，经过串入并出电路的转换，当接收完一个字符之后，数据就从接收移位寄存器传送到数据输入缓冲器，等待CPU读取。
- 数据输出寄存器。当CPU输出数据时，先送到数据输出缓冲器，然后，数据由输出寄存器传到发送移位寄存器，经过并入串出电路转换一位一位地通过输出传输线送到外设。
- 状态寄存器。状态寄存器用来存放外设运行的状态信息，CPU通过访问这个寄存器来了解某个外设的状态，进而控制外设的工作，以便与外设进行数据交换。
- 控制寄存器。串行接口中有一个控制寄存器，CPU对外设设置的工作方式命令、操作命令都存放在控制寄存器中，通过控制寄存器控制外设运行。

串行接口基本工作原理：串行发送时，CPU通过数据总线把8位并行数据送到数据输出寄存器，然后送给并行输入/串行输出移位寄存器，并在发送时钟和发送控制电路控制下通过串行数据输出端一位一位串行发送出去。起始位和停止位是由串行接口在发送时自动添加上去的。串行接口发送完一帧后产生中断请求，CPU响应后可以把下一个字符送到发送数据缓冲器。

串行接口基本工作原理：串行接收时，串行接口监视串行数据输入端，并在检测到有一个低电平（起始位）时就开始一个新的字符接收过程。串行接口每接收到一位二进制数据位后就使接收移位寄存器(即串

行输入-并行输出寄存器)左移一次;连续接收到一个字符(字节)后将其并行传送到数据输入寄存器, 并产生中断促使CPU从中取走所接收的字符。

◇UART(Universal Asynchronous Receiver/Transmitter



硬件 UART的结构

UART出错的三种标志:

- 奇偶错误, 1的个数不对。
- 帧错误, 接收时钟和发送时钟的频率相差太大。
- 溢出错, 缓存器满。

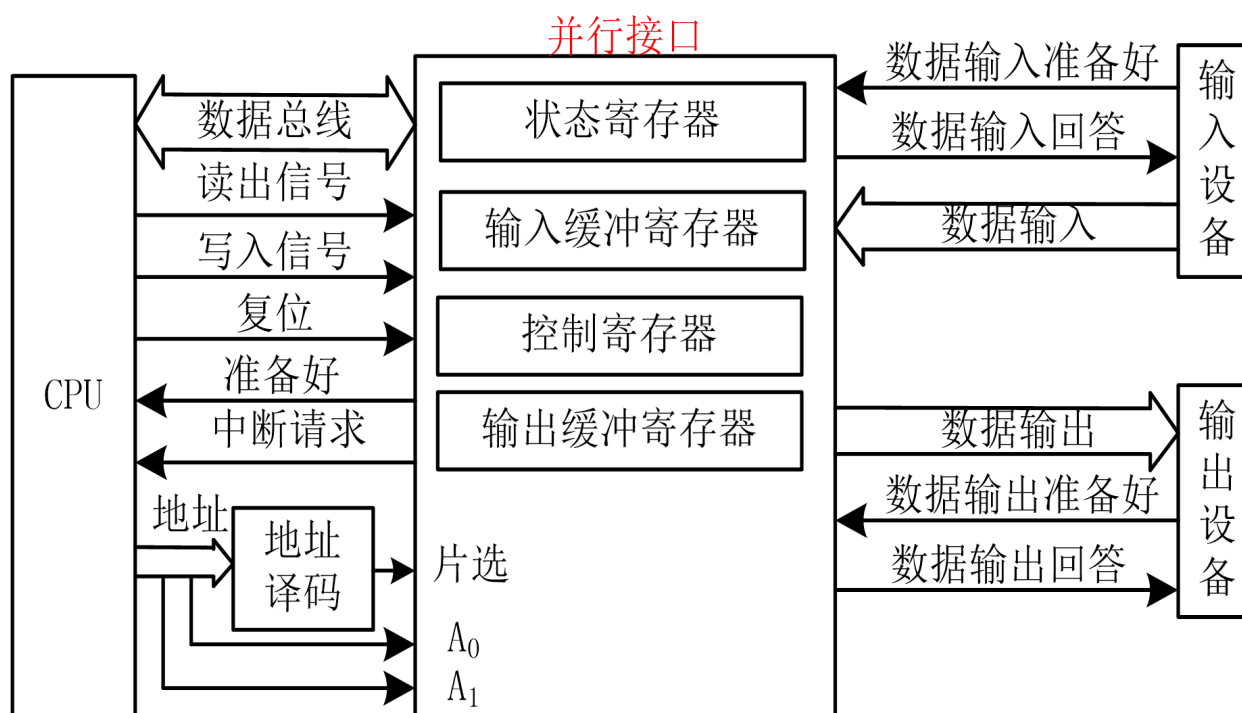
并行通信

并行接口

并行接口: 实现并行通信的接口电路。

分类: 输入并行接口、输出并行接口和输入/输出并行接口。

并行通信以同步方式传输, 其特点是: 传输速度快;硬件开销大;适合近距离传输。



并行接口的传输信息包括：状态信息、控制信息、数据信息。

- 状态信息：状态信息表示外设当前所处的工作状态。例如，准备好信号“READY”=1表示输入接口已经准备好，可以和CPU交换数据；忙信号“BUSY”=1表示接口正在传输信息，CPU需要等待。
- 控制信息：控制信息是由CPU发出的，用于控制外设接口的工作方式以及外设的启动和复位等。
- 数据信息：CPU与并行接口交换的主要内容。

并行接口电路组成

- 输入缓冲寄存器。输入数据缓冲器主要功能是负责接收设备送来的数据，CPU通过读操作指令(IN) MOVX A, @DPTR执行读操作，从输入数据缓冲器读取数据。
- 输出缓冲寄存器。输出数据缓冲器主要功能是负责接收CPU送来的数据，如果设备处于空闲状态，则从输出数据缓冲器取走数据，接口通知CPU进行下一次输出操作。
- 状态寄存器。状态寄存器用来存放外设运行状态信息，CPU通过访问状态寄存器来了解外设状态，进而控制外设的工作。
- 控制寄存器。并行接口中有一个控制寄存器，CPU对外设设置的工作方式命令、操作命令都存放在控制寄存器中，通过控制寄存器控制外设的运行。
- 数据信息。CPU与并行接口交换的主要内容。

串行接口

IAP15W4K58S4单片机具有4个采用UART工作方式的全双工串行通信接口(串口1,串口2,串口3,串口4)。

每个串口由2个数据缓冲器、1个(输入)移位寄存器、1个串行控制寄存器和一个波特率发生器等组成。

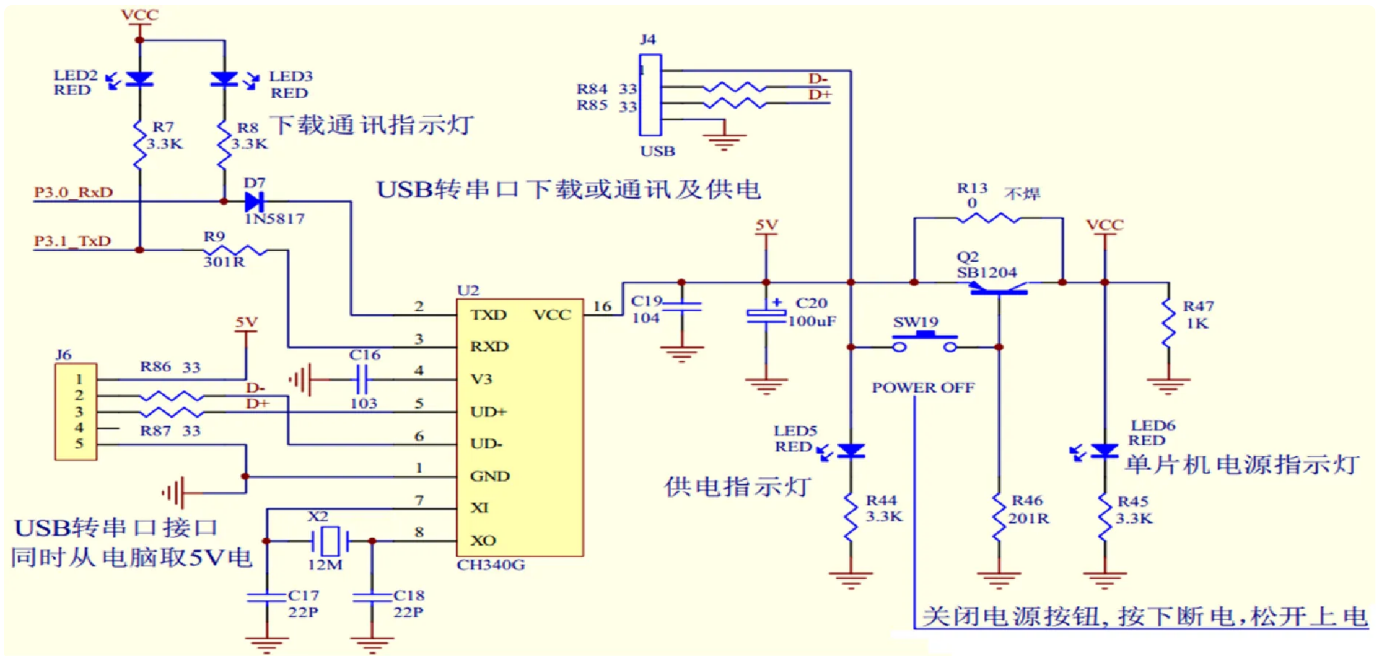
每个串口的数据缓冲器由串行接收缓冲器和发送缓冲器构成，它们在物理上独立，既可以接收数据也可以发送数据，还可以同时发送和接收数据。

接收缓冲器只能读出，不能写入，而发送缓冲器则只能写入，不能读出。它们共用一个地址号。

RS232串行通信接口

RS-232是早期为公用电话网络数据通信而制定的标准。其逻辑电平与TTL/CMOS电平完全不同。逻辑“0”规定为+5~+15V之间，逻辑“1”规定为-5~-15V之间。由于RS-232发送和接收之间有公共地，传输采用非平衡模式，因此共模噪声会耦合到信号系统中，标准中建议的最大通信距离为15米。

计算机串口是RS-232电平，而单片机串口是TTL电平。因此，须用电路实现TTL电平和RS-232电平转换。常用电平转换芯片是MAX232(或兼容芯片SP3232等)，其用单电源(+5V)供电，RS232电平由内部电荷泵产生。



常见的USB转串口芯片有CH340T、CP210x等。CH340与单片机的接口电路见图4-28。USB转串口电路只是实现USB传输协议和UART传输协议之间的转换，对于用户而言，程序设计上没有本质的改变，相当于一个串口使用。唯一需要注意的是，在计算机上编程选择串口的时候，串口号的选择要正确。

RS485串行通信接口

RS-485串行数据接口是为弥补RS-232通信距离短（15米）、速率低等缺点而产生的。在RS-422基础上制定的标准，增加了多点、双向通信能力，即允许多个发送器连接到同一条总线上，同时增加了发送

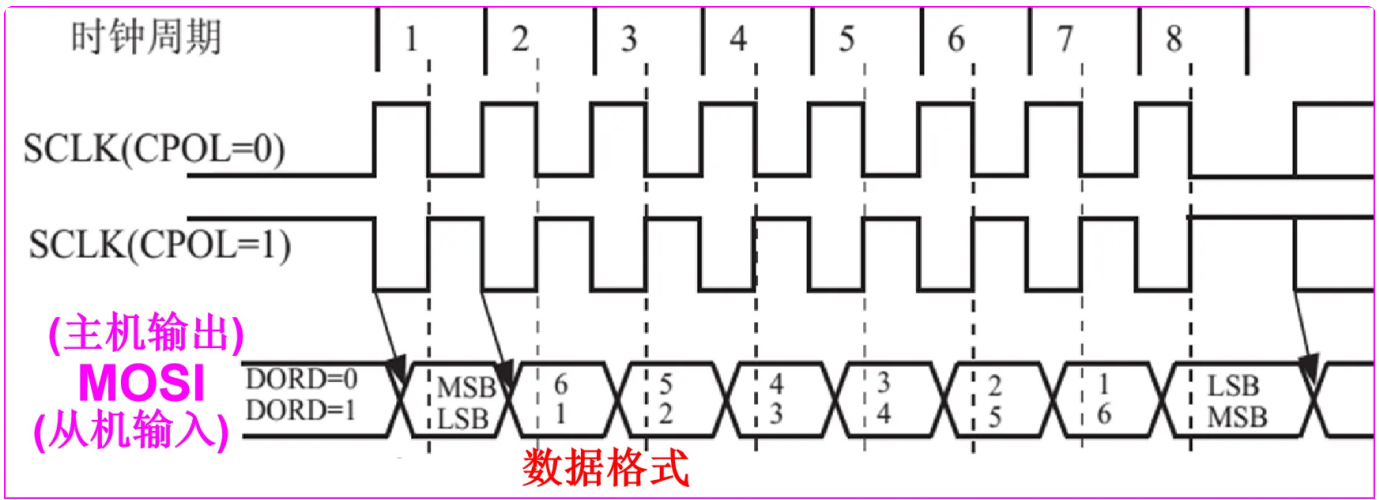
器的驱动能力和冲突保护特性。RS-485标准只规定了平衡发送器和接收器的电特性，而没有规定接插件、传输电缆和应用层通信协议。

RS-485数据信号采用差分传输方式，也称作平衡传输，它使用一对双绞线，将其中一线定义为A，另一线定义为B。A、B之间(A-B)的正电平在+2V~+6 V，表示逻辑状态“1”；负电平在-2V~-6 V，表示逻辑状态“0”。RS-485标准的最大传输距离约为1200米，最大传输速率为10Mbps。

SPI通信接口

IAP15W4K58S4集成了串行外设接口(Serial Peripheral Interface, 简称SPI)。SPI接口既可以和其他微处理器通信，也可以与具有SPI兼容接口的器件，如存储器、A/D转换器、D/A转换器、LED或LCD驱动器等

等进行同步通信。SPI接口有两种操作模式：主模式和从模式



SPI的核心是一个8位移位寄存器和数据缓冲器，数据可以同时发送和接收。在SPI数据的传输过程中，发送和接收的数据都存储在缓冲器中。对于主模式，若要发送一个字节数据，只需将这个数据写到SPDAT寄存器中。主模式下/SS信号不是必须的。在从模式下，必须在/SS信号变为有效并接收到合适的时钟信号后，方可进行数据的传输。在从模式下，如果一个字节传输完成后，/SS信号变为高电平，这个字节立即被硬件逻辑标志为接收完成，SPI接口准备接收下一个数据。任何SPI控制寄存器的改变将复位SPI接口，并清除相关寄存器。

MOSI (Master Out Slave In, 主出从入) 主器件的输出和从器件的输入，用于主器件到从器件的串行数据传输。根据SPI规范，多个从机共享一根MOSI信号线。在时钟边界的前半周期，主机将数据放在MOSI信号线上，从机在该边界处获取该数据。

MISO (Master In Slave Out, 主入从出) 从器件的输出和主器件的输入。用于实现从器件到主器件的数据传输。SPI规范中，一个主机可连接多个从机，因此，主机

MISO信号线会连接到多个从机上，或者说，多个从机共享一根MISO信号线。当主机与一个从机通信时，其他从机应将其MISO引脚驱动置为高阻状态。

SCLK (SPI Clock, 串行时钟信号) 串行时钟信号是主器件的输出和从器件的输入, 用于同步主器件和从器件之间在MOSI和MISO线上的串行数据传输。当主器件启动一次数据传输时, 自动产生8个SCLK时钟周期信号给从机。在SCLK的每个跳变处 (上升沿或下降沿) 移出一位数据。一次数据传输可传输一个字节的的数据。

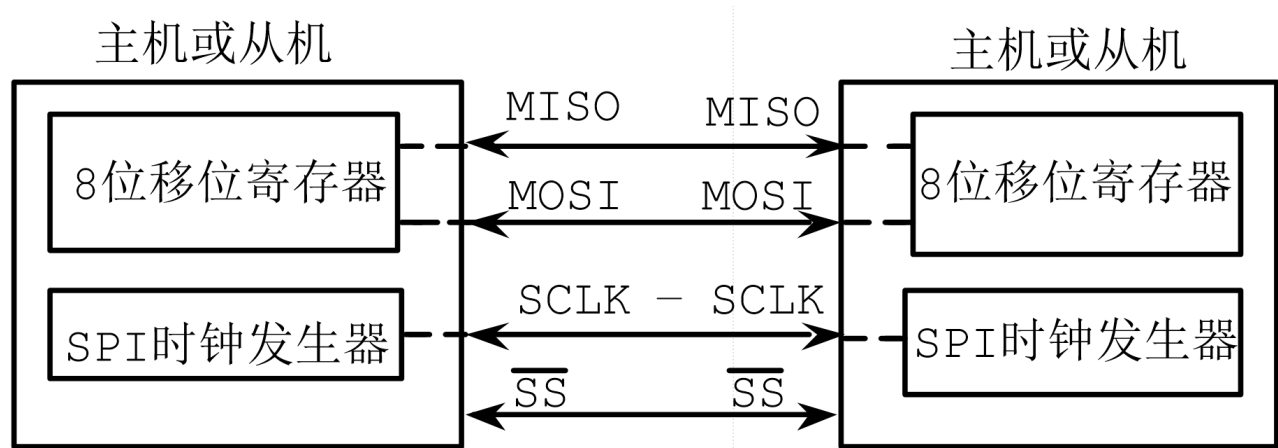
$\overline{SS}$ (Slave Select, 从机选择信号) 这是一个输入信号。主器件用它来选择处于从模式的SPI模块。在主模式下, SPI接口就是主机(只允许一个主机), 不存在主机选择问题, 该模式下/ $\overline{SS}$ 不是必须的。主模式下通常将主机的/ $\overline{SS}$ 引脚通过10k Ω 电阻上拉至高电平。每个从机的/ $\overline{SS}$ 接主机的I/O口, 由主机控制电平高低, 以便主机选择从机。在从模式下, 不论发送还是接收, / $\overline{SS}$ 信号必须有效。因此在一次数据传输开始之前必须将/ $\overline{SS}$ 拉为低电平。SPI主机可用I/O口选择一个SPI器件(使/ $\overline{SS}$ =0)作为当前从机。

IAP15W4K58S4单片机SPI接口的数据通信方式有3种:

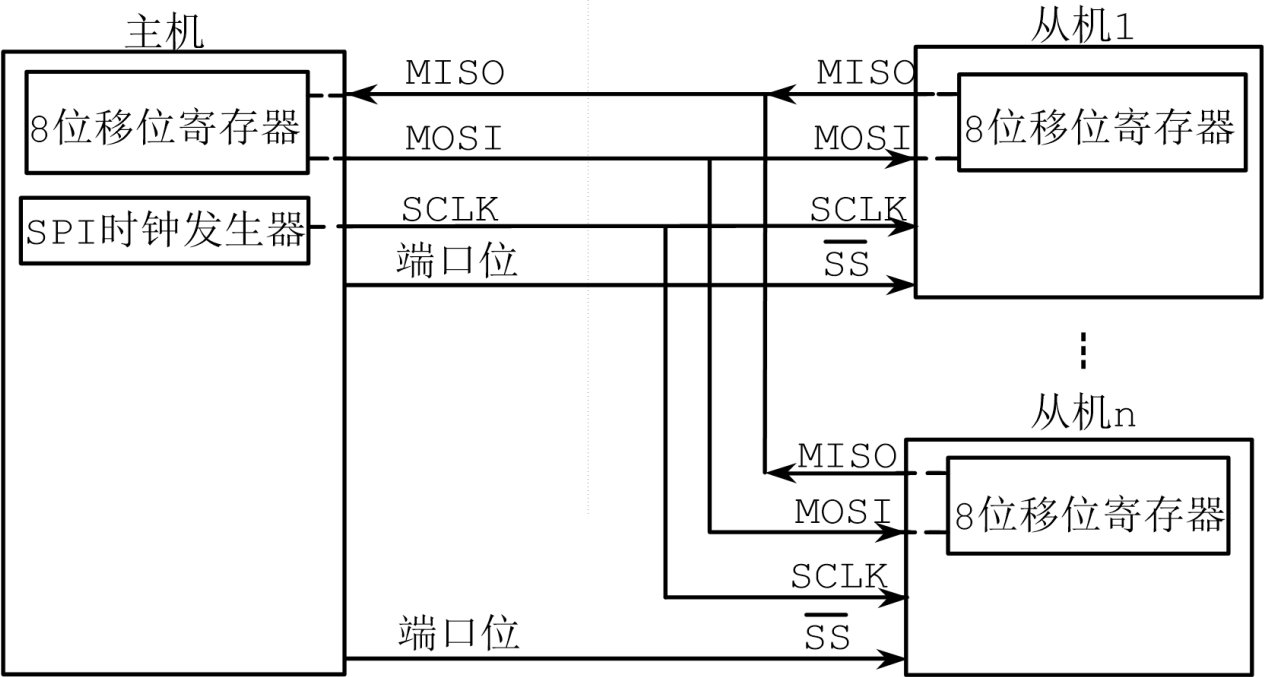
- 单主机-单从机方式



- 双器件方式 (器件可互为主机和从机)



- 单主机-多从机方式



▼ SPI协议详解

SPI (Serial Peripheral Interface, 串行外设接口) 是一种用于微控制器 (MCU) 与外部设备 (如传感器、存储器、显示器等) 之间通信的串行数据传输协议。SPI协议由Motorola公司于1980年代提出, 广泛用于低至中速的短距离通信。

SPI协议的基本原理

SPI是一种**全双工** (Full-Duplex) 同步串行通信协议, 意味着数据可以同时两个方向上传输。它采用了**主从** (Master-Slave) 架构, 其中主设备控制通信过程, 发起时钟信号, 而从设备响应。SPI协议使用4根基本的信号线, 分别是:

1. **MOSI (Master Out Slave In)** : 主设备发送给从设备的数据线。
2. **MISO (Master In Slave Out)** : 从设备发送给主设备的数据线。
3. **SCK (Serial Clock)** : 时钟信号, 由主设备提供, 用于同步数据传输。
4. **SS (Slave Select)** : 从设备选择信号, 当主设备要与某个从设备通信时, SS信号会激活该从设备。

SPI的工作流程

在SPI协议中, 主设备会生成时钟信号SCK, 并通过MOSI线将数据发送给从设备。同时, 从设备也会通过MISO线将数据返回给主设备。以下是SPI的典型通信步骤:

1. 主设备激活SS信号, 选择一个从设备。
2. 主设备通过SCK发送时钟信号, 定义了数据传输的速率。
3. 主设备从MOSI线发送数据, 每个时钟周期传送一位数据。
4. 在每个时钟周期的同时, 从设备将数据通过MISO线返回给主设备。
5. 通信结束后, 主设备取消SS信号, 通信终止。

SPI的工作模式

SPI协议的灵活性表现在它可以选择不同的时钟极性 (CPOL) 和时钟相位 (CPHA), 这决定了数据传输的具体时序。根据时钟的不同配置, SPI有四种工作模式 (Mode 0-3) :

- **CPOL (时钟极性)** : 时钟信号的空闲状态。它决定了SCK空闲时是高电平还是低电平。
 - CPOL = 0: 时钟空闲时是低电平。
 - CPOL = 1: 时钟空闲时是高电平。
- **CPHA (时钟相位)** : 数据采样的时机。它决定了数据在SCK时钟信号的哪个边沿 (上升沿

或下降沿) 进行采样。

- CPHA = 0: 数据在时钟的第一个边沿 (上升沿) 采样。
- CPHA = 1: 数据在时钟的第二个边沿 (下降沿) 采样。

SPI的四种模式组合如下:

Mode	CPOL	CPHA
Mode 0	0	0
Mode 1	0	1
Mode 2	1	0
Mode 3	1	1

SPI的优点和缺点

优点

1. **高速数据传输**: SPI协议支持较高的传输速率, 适用于需要较高带宽的应用。
2. **全双工通信**: 可以同时数据进行接收和发送, 提高通信效率。
3. **简单的硬件连接**: 只需要四根线 (MOSI、MISO、SCK、SS), 硬件设计简洁。
4. **灵活性**: 可以通过调整时钟极性和相位来适应不同外设的需求。

缺点

1. **需要较多的引脚**: 对于每个从设备, 主设备需要额外的SS引脚, 这在设备较多时会占用较多的GPIO引脚。
2. **点对点通信**: SPI协议本身不支持多主机系统, 如果要与多个主设备通信, 需要额外的硬件支持。
3. **没有错误检测**: SPI协议本身没有内建的错误检测机制, 传输过程中需要额外的手段来确保数据的正确性。

SPI与其他通信协议的比较

1. 与I2C的比较:

- I2C是另一种常见的串行通信协议, 使用两根线 (SCL和SDA)。与SPI相比, I2C在设备数量上更为灵活, 可以连接多个设备, 但其传输速率较低, 且支持的通信距离较短。
- SPI通常比I2C速度更快, 且支持全双工通信, 但需要更多的引脚。

2. 与UART的比较:

- UART是异步串行通信协议，通常用于远距离通信，而SPI是同步通信协议，适用于近距离高速数据传输。
- UART只支持单工（half-duplex）或半双工（half-duplex）通信，而SPI支持全双工通信。

总结

SPI协议是一种高效、简单、灵活的串行通信协议，适合用于主设备与多个从设备之间的高速数据交换。它的应用非常广泛，尤其在嵌入式系统中，用于与传感器、存储设备、显示器等外设的通信。虽然它的硬件连接要求较多，但其高速、全双工的特点使其在很多场合都非常有优势。

▼ I2C协议详解

I2C（Inter-Integrated Circuit）是一种常用于短距离通信的串行总线协议，由飞利浦公司（现为NXP）于1980年代初期开发，广泛应用于微控制器、传感器、显示器等设备之间的通信。I2C协议是多主机、多从机的同步串行通信协议，具有较低的硬件要求，且支持多个设备共享同一总线。以下是I2C协议的详细介绍：

1. 基本结构

I2C通信通过两条线进行数据传输：

- **SDA（Serial Data Line）**：串行数据线，用于传输数据。
- **SCL（Serial Clock Line）**：串行时钟线，用于同步数据传输。

I2C协议采用**主从模式**，其中：

- **主设备（Master）**：负责发起通信，控制时钟（SCL线），并决定何时开始与哪个从设备进行数据交换。
- **从设备（Slave）**：响应主设备的命令，按照主设备的要求传输数据。

2. 通信过程

I2C的通信过程基于“地址-数据”的形式，并且采用应答机制（ACK）来确保数据传输的完整性。其基本步骤如下：

2.1 起始信号（START）

通信开始时，主设备首先通过在SCL保持高电平时将SDA从高拉低，产生一个**起始信号**（START condition）。这是通信的标志，表示总线被占用。

2.2 设备地址和读/写标志

主设备发出一个7位的**设备地址**，并在其后附带一个**读/写标志**：

- **写操作（0）**：表示主设备将数据发送到从设备。
- **读操作（1）**：表示主设备从从设备接收数据。

设备地址后面是一个应答位（ACK）。从设备根据其地址进行识别，并向主设备发送应答信号。

2.3 数据传输

- **数据字节**：I2C每次传输8位（1字节）数据。每发送完一个字节后，接收方会发送一个应答

位（ACK）以确认数据的接收。

- **数据传输方向**：在写操作中，数据由主设备发送给从设备；在读操作中，数据由从设备发送给主设备。

2.4 停止信号（STOP）

通信结束时，主设备发出一个**停止信号**（STOP condition），这是通过在SDA从低电平跳变到高电平时，在SCL保持高电平时产生的。

2.5 重复启动信号（Repeated START）

有时，主设备在不中断通信的情况下，重新发起另一个通信。此时可以使用**重复启动信号**（Repeated START），它不产生停止信号，而是重新产生一个起始信号。重复启动信号使得主设备可以与多个从设备进行通信。

3. 数据传输格式

一个典型的数据传输帧包含以下几个部分：

1. **起始信号**（START condition）
2. **设备地址和读/写标志**：7位设备地址和1位读/写标志（8位总长）
3. **数据字节**：每个数据字节为8位，并且每个字节后跟着一个应答位（ACK）。
4. **停止信号**（STOP condition）

4. I2C的地址分配

- **7位地址模式**：I2C协议默认使用7位设备地址。设备地址范围从0x00到0x7F（0到127），其中0x00到0x07通常是保留地址。
- **10位地址模式**：I2C也支持10位地址模式，这时设备地址的范围是0x000到0x3FF。

5. 多主机模式

I2C允许多个主设备同时连接到总线上，这意味着不同的主设备可以根据总线的空闲状态控制通信。但这要求对总线的访问进行严格的协调，以避免总线冲突。

6. I2C的优点

- **简洁性**：I2C只使用两条线（SDA和SCL），减少了硬件设计的复杂性。

- **多主从支持**：可以连接多个主设备和多个从设备。
- **支持低速通信**：适用于短距离和低速的数据传输。
- **较低的成本**：硬件实现简单且成本低。

7. I2C的缺点

- **速度较低**：I2C的传输速率通常在100kHz（标准模式）、400kHz（快速模式）或更高的1MHz（高速模式）之间，相对于其他协议（如SPI），速度较慢。
- **传输距离有限**：I2C适用于短距离通信，通常传输距离不能超过几米。
- **总线拥塞问题**：如果多个设备在同一总线上进行通信，可能导致总线冲突和拥塞，影响性能。

8. I2C与其他通信协议的比较

- **与SPI相比**：SPI（Serial Peripheral Interface）是另一种常见的串行通信协议。I2C具有更少的引脚（两条线），而SPI需要更多的引脚（通常至少四条线）。I2C适用于低速、短距离通信，而SPI适用于高速、长距离通信。
- **与UART相比**：I2C适用于多设备之间的通信，而UART通常用于点对点的通信。

9. I2C的应用

I2C被广泛应用于：

- 微控制器与外设（如传感器、显示器、EEPROM等）之间的通信。
- 嵌入式系统中的通信总线。
- 低速数据传输和低功耗设备。

总结

I2C协议是一种简单、高效的串行通信协议，广泛应用于低速和短距离设备之间的通信。它通过共享两条线实现多个设备之间的通信，具有较低的硬件要求，适用于多设备系统，但也有一定的速度和距离限制。在实际应用中，I2C在许多嵌入式系统中起到了关键作用。

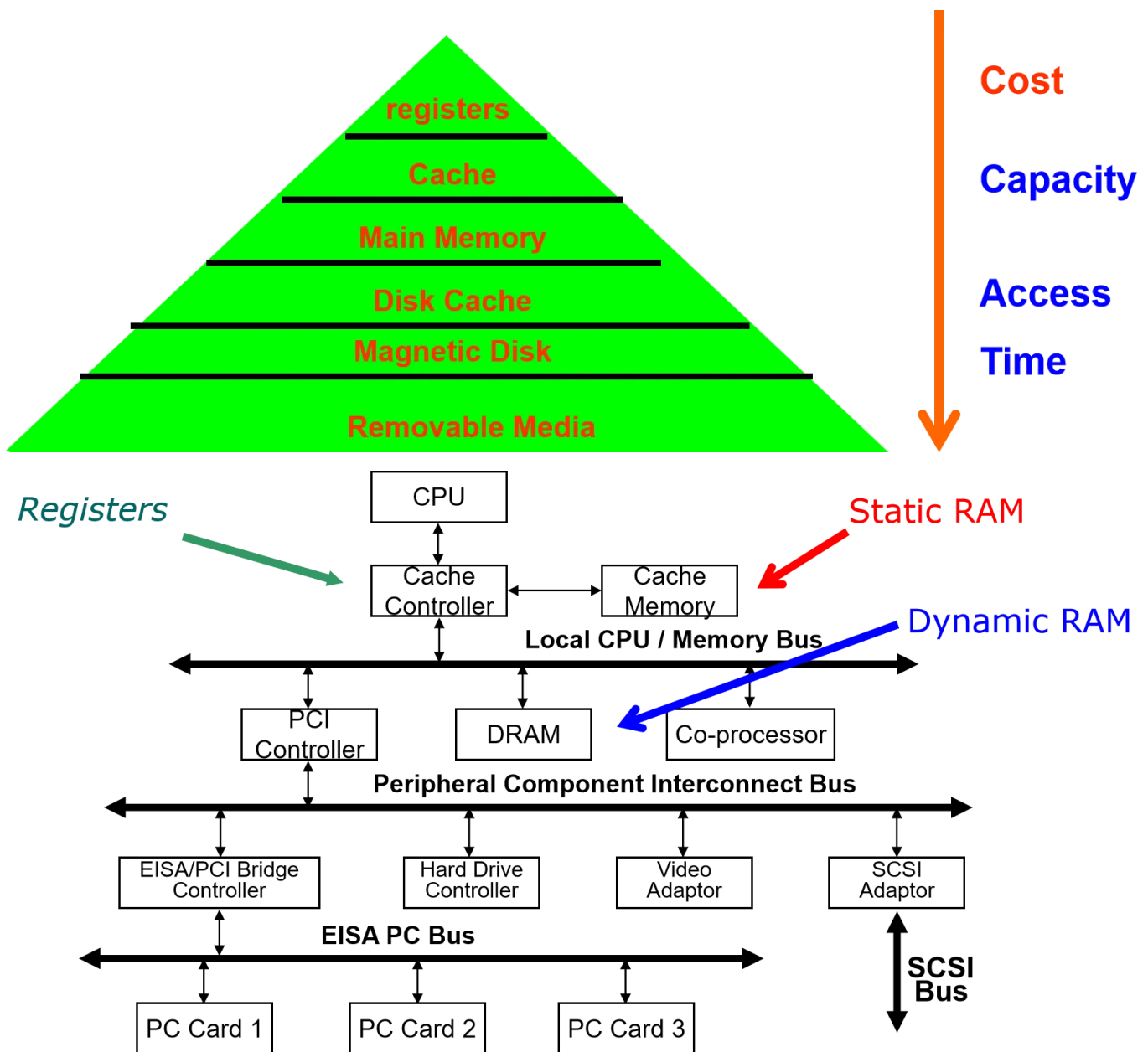
第六章 存储器

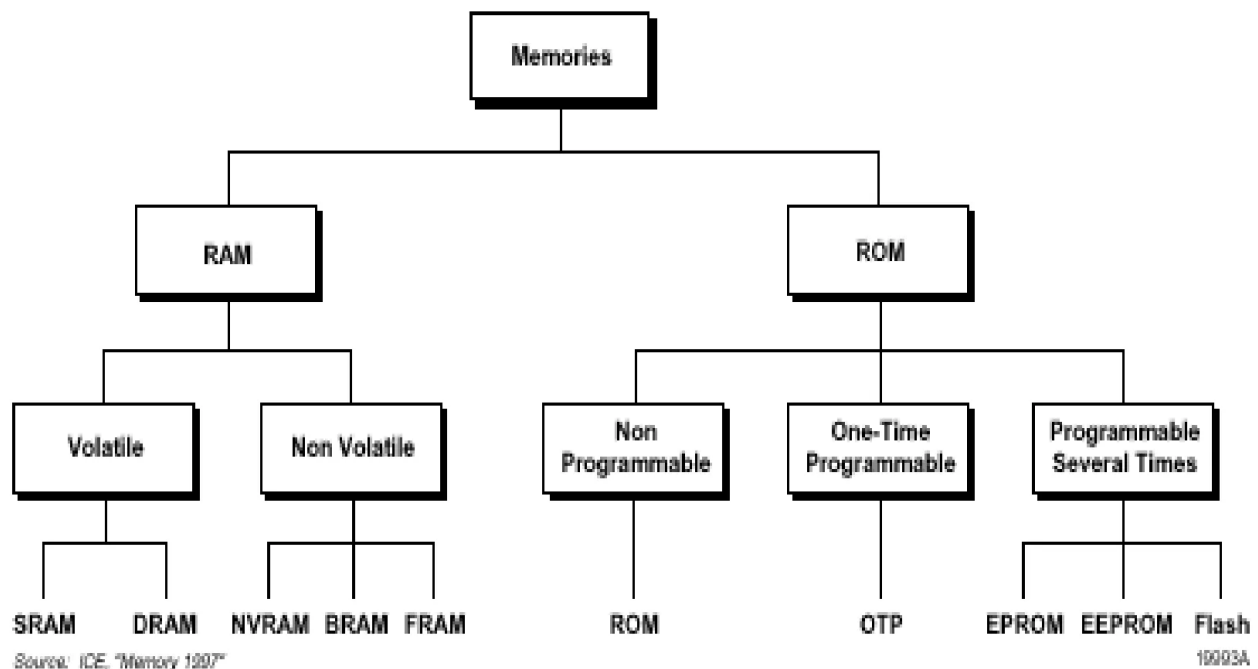
CPU – Memory Interface usually consists of:

- unidirectional address bus
- bidirectional data bus
- read control line
- write control line
- ready control line
- size (byte, word) control line

Memory access involves a memory bus transaction

- read: set address, read control signal and size signal, copy data when ready is set by memory
- write: set address, data, write control signal and size, done when ready is set





Key Design Metrics:

1. Memory Density (number of bits/um?) and size
2. Access Time(time to read or write) and Throughput
3. Power Dissipation

➤ RAM

- ✓ Read/write
- ✗ Volatile
- ✓ Faster access time

➤ Variants

- ☐ SRAM
- ☐ DRAM

➤ Application

- ☐ Variables
- ☐ Dynamic memory allocation
- ☐ Heaps, stacks

➤ ROM

- ✗ Read only
- ✓ Non-Volatile
- ✗ Slower

➤ Variants

- ☐ PROM, EPROM
- ☐ EEPROM, FLASH

➤ Application

- ☐ Programs
- ☐ Constants
- ☐ Codes, e.t.c

DRAM: Dynamic Random Access Memory

- upside: very dense (1 transistor/capacitor per bit) and inexpensive

- downside: requires refresh and often not the fastest access times
- often used for main memories

SRAM: Static Random Access Memory

- upside: fast and no refresh required
- downside: not so dense (6 transistors per cell) and not so cheap
- often used for caches

ROM: ReadOnly Memory (Flash, EPROM, EEPROM)

- Upside: nonvolatile
- Downside: Slow to write to
- often used for bootstrapping, implementing functions (SQRT)